

The Maximum Clique Interdiction Problem

Fabio Furini* Ivana Ljubić† Sébastien Martin‡ Pablo San Segundo§

Abstract

Given a graph G and an interdiction budget k , the *Maximum Clique Interdiction Problem* asks to find a subset of at most k vertices to remove from G so that the size of the maximum clique in the remaining graph is minimized. This problem has applications in many areas, such as crime detection, prevention of outbreaks of infectious diseases and surveillance of communication networks. We propose an integer linear programming formulation of the problem based on an exponential family of *Clique-Interdiction Cuts* and we give necessary and sufficient conditions under which these cuts are facet-defining. Our new approach provides a useful tool for analyzing the resilience of (social) networks with respect to clique-interdiction attacks, i.e., the decrease of the size of the maximum clique as a function of an incremental interdiction budget level. On a benchmark set of publicly available instances, including large-scale social networks with up to one hundred thousand vertices and three million edges, we show that most of them can be analyzed and solved to proven optimality within short computing time.

Keywords: Combinatorial Optimization , Interdiction Problems , Maximum Clique , (Social) Network Analysis , Most Vital Vertices.

1 Introduction

Decision makers are often faced with the question of identifying the *most vital* (also called *most vulnerable* or *most critical*) part of a network, which corresponds to a subset of vertices (or edges) of limited size, whose malfunctioning prevents the functionality of the network as a whole. Depending on the crucial property that needs to be maintained (or achieved) in the network, different vertices may be perceived as the most important ones. In some applications, we are looking for the most influential vertices (when it comes to spreading information in the network, see, e.g. [Kempe et al. \(2005\)](#)). In other applications, we might ask for the most critical vertices that may affect or destroy connectivity of the network, see, e.g. [Furini et al.](#); [Lalou et al. \(2018\)](#).

In this work we are concerned with the *cohesiveness property* of a (social) network. Within a network, there exist groups of individuals who interact with each other to such an extent that they could be perceived as separate entities. Such “tightly knit” groups are frequently identified

*Université Paris-Dauphine, Paris, France, fabio.furini@dauphine.fr

†ESSEC Business School, Cergy-Pontoise, France, ivana.ljubic@essec.edu

‡Université de Lorraine, Metz, France, sebastien.martin@univ-lorraine.fr

§Center of Automation and Robotics, Univ. Politécnica de Madrid, Spain, pablo.sansegundo@upm.es

using the notion of a *clique*, i.e., a subset of vertices that are pairwise connected. Networks with large cliques are considered cohesive, as the cliques model the archetypal structural pattern of the cohesion idea (Moody and Coleman, 2015; Moody and White, 2003). The *Maximum Clique Problem* in a graph $G = (V, E)$ (or a network) is a well-studied problem in Graph Theory and Operations Research. Its solution characterizes one of the most essential graph properties known as the *clique number* $\omega(G)$ of a graph, i.e., the size of a maximum clique in G . The clique number is also used as one of the measures of cohesiveness of a network Borgatti et al. (2018).

In this article, we study the problem of identifying a most vital subset of vertices with respect to the clique number, which is formally defined as follows: Given a graph G and an interdiction budget $k \geq 1$, the *Maximum Clique Interdiction Problem* (CIP) asks to find a subset of at most k vertices to remove from G so that the clique number in the remaining graph is minimized. The set of interdicted vertices is called an *optimal interdiction strategy*, or equivalently, a set of the *most vital vertices* of the graph with respect to the clique number.

In the context of crime detection and prevention, large cliques have been identified as potential origins of catastrophic events such as terrorist or hacker attacks (Berry et al., 2004; Sageman, 2004), as well as sources of outbreaks of sexually transmitted diseases (Rothenberg et al., 1996). In the analysis of terrorist networks (see, e.g., Chen et al., 2004; Sampson and Groves, 1989), cliques are used to model communities due to their ability to encode the interactions of groups of individuals who are all close friends with one another. Sageman Sageman (2004) argues that cliques commonly produce social cohesion and a collective identity due to the interactions that foster solidarity and trust. Large cliques in a terrorist or crime network are capable of designing sophisticated large-scale operations like those of the 9/11 attacks, the series of coordinated terrorist attacks in Paris in 2014 and 2015, and numerous devastating coordinated attacks throughout Europe, Middle East, Africa and Asia, see (2017 Terrorist Attacks) for a detailed map on recent attacks. More recent examples of security games in terrorist networks can be found e.g., in Guo et al. (2016); Wang et al. (2016). Hence, the cohesiveness of a terrorist/criminal network is one of its important features that needs to be carefully monitored and potentially reduced.

Applications of the CIP can also be found in other areas. For example, in modern telecommunication networks, Service Functions (SFs) such as firewalls, deep packet inspections (DPIs), web proxies, media gateways, etc. are used as middleboxes on the Service Provider Networks. Network Function Virtualization permits to virtualize the SFs so that using Software Defined Networks (SDNs), SFs can be disassociated from the physical elements of the network and installed as a software. This allows for a centralized control of network resources in which a centralized controller has visibility over the entire network, and has a complete view of the network topology (Kim and Feamster, 2013). An optimal CIP strategy in an SDN network identifies a set of vertices where SFs have to be installed so as to monitor and prevent suspicious activities while keeping the size of the remaining non-monitored cliques as small as possible.

In Figure 1, we provide a synthetic example graph of 12 vertices. In this graph, the clique number is 4 and there are four maximum cliques, i.e., $K_1 = \{v_1, v_2, v_3, v_4\}$, $K_2 = \{v_8, v_9, v_{10}, v_{11}\}$,

$K_3 = \{v_1, v_3, v_4, v_5\}$ and $K_4 = \{v_6, v_8, v_9, v_{11}\}$. If the interdiction budget k is equal to 2, an optimal interdiction strategy is shown in Figure 2. This strategy suggests that the pair of vertices depicted in black is critical for maintaining the cohesiveness of the graph. After removing vertices $\{v_4, v_{11}\}$, the clique number becomes 3 and there are six maximum cliques, e.g., $\{v_8, v_9, v_{10}\}$ or $\{v_6, v_8, v_9\}$. It is important to notice that the interdiction strategy is often not unique. For instance, there are $3 \cdot 3$ optimal interdiction strategies obtained by choosing one vertex from the set $\{v_1, v_3, v_4\}$ and one vertex from the set $\{v_8, v_9, v_{11}\}$. Furthermore, we can also observe in the graph of Figure 1 that the classical centrality measures (like *degree*, *closeness*, *betweenness* or *eigenvector centrality*) rank the vertices $\{v_5, v_6\}$ as the most central ones. These two vertices are not the most critical ones for cohesiveness (i.e., the vertices necessary to preserve the clique number of the graph). On the contrary, among the 9 optimal CIP strategies (for $k = 2$), none contains v_5 nor v_6 . The vertices identified as the most critical ones for the cohesiveness are instead the ones belonging to the intersection of the two maximum cliques K_2 and K_4 and to the intersection of the other two maximum cliques K_1 and K_3 .

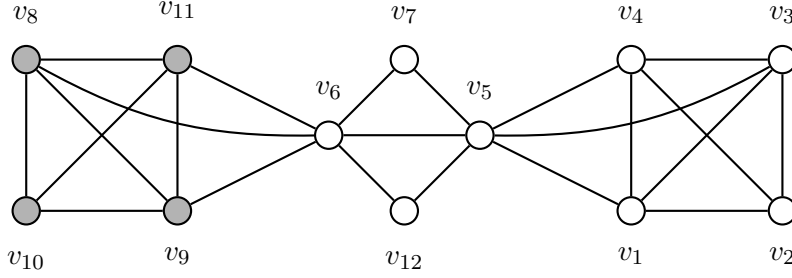


Figure 1: An example graph G for the CIP. Before interdiction, the clique number is $\omega(G) = 4$. One of the four maximum cliques is shown in grey, i.e., the clique $K_2 = \{v_8, v_9, v_{10}, v_{11}\}$.

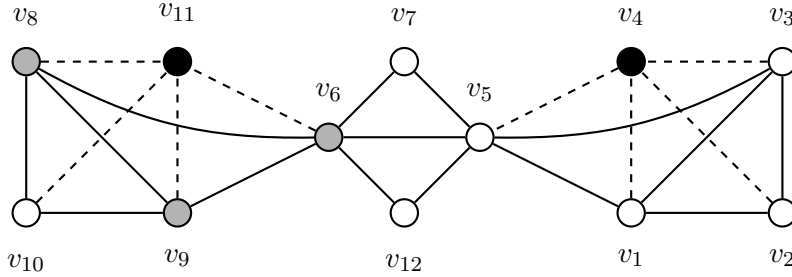


Figure 2: The vertices of an optimal interdiction strategy of the CIP with a budget $k = 2$ are shown in black, and one of the remaining maximum cliques of size 3 is shown in grey, i.e., the clique $\{v_6, v_8, v_9\}$. Dashed lines represent the edges which are incident to the interdicted vertices, as they cannot be used to build cliques in $G[V \setminus \{v_4, v_{11}\}]$.

The CIP under investigation belongs to a family of Interdiction Problems (see e.g., Fischetti et al. (2018); Tang et al. (2016)) which are a special class of *Bilevel Optimization Problems*. The latter ask for a solution of a two-level optimization problem in which the first level decisions affect the second-level optimization problem. Bilevel optimization problems are also seen as two-player Stackelberg games in which decisions of the first- and second-level are commonly denoted as *leader's*, respectively *follower's* decisions (see, e.g. a recent survey on bilevel optimization in Dempe (2018)). In our setting, the leader first determines a subset of vertices V' to be removed

from the network using a limited interdiction budget. After that, the follower observes the set V' and chooses a maximum clique in the remaining graph. Being able to anticipate the optimal follower's solution for each of the possible vertex-removal strategies, the goal of the leader is to choose a strategy V' that results in the worst possible outcome for the follower. More precisely, the goal of the leader is to find a best vertex-removal strategy so that the size of the maximum clique on the remaining graph is *minimized*.

We refer the interested reader to [Fischetti et al. \(2017\)](#); [Lozano and Smith \(2017\)](#) for the most recent exact approaches in bilevel combinatorial optimization. In the literature, different classical combinatorial optimization problems have been studied in the bilevel context. Some of the most prominent examples include studies in which the follower solves the knapsack problem, the shortest-path problem, or the maximum flow problem on an interdicted network [Fischetti et al. \(2018\)](#); [Ratliff et al. \(1975\)](#); [Tang et al. \(2016\)](#); [Wood \(2010\)](#); [Cormican et al. \(1998\)](#). Interdiction problems on networks have been widely studied in the stochastic setting as well, see, e.g., [\(Song and Shen, 2016\)](#). Some relevant applications of interdiction problems include the firefighting problem ([Adjashvili et al., 2017](#); [Baggio, 2016](#); [García-Martínez et al., 2015](#)), the control of infections in hospitals ([Assimakopoulos, 1987](#)), the allocation of protective resources in shortest-path networks ([Cappanera and Scaparra, 2011](#)), the protection and analysis of supply chain disruptions ([Snyder et al., 2016](#)), or the detection of drug smuggling ([Washburn and Wood, 1995](#)). Finally, knapsack interdiction problems are studied in ([Fischetti et al., 2018](#); [Caprara et al., 2016](#)).

Article Contribution To the best of our knowledge, this article provides the first comprehensive theoretical and computational study on how to find the *k most vital vertices of a graph* with respect to its clique number. We provide a computationally efficient framework for solving the problem in large-scale realistic networks. The overall framework is constructed using building blocks that are derived from both, theoretical and algorithmic results. These results can be summarized as follows:

- The decision version of the CIP is Σ_2^P -complete (see [Rutenburg \(1991, 1994\)](#); [Ko and Lin \(1995\)](#)), but for a special class of graphs being a union of disjoint cliques, we show that the CIP can be solved in polynomial time. We also provide a closed formula for calculating the optimal solution for this special case.
- The latter result is then exploited for deriving a tight combinatorial lower bound $\ell_{\min}$ for solving the problem on general graphs.
- We provide conditions under which redundant vertices can be removed from the graph. For the efficient reduction of the input graph, we show that the tight value of $\ell_{\min}$ plays a crucial role.
- Starting from a Bilevel Integer Programming model, we then derive a single-level reformulation based on an exponential family of *Clique-Interdiction Cuts*.
- We provide necessary and sufficient conditions under which clique-interdiction cuts are

facet defining and we propose a fast lifting procedure.

- For the separation of clique-interdiction cuts, we develop a specially tailored combinatorial algorithm derived from one of the state-of-the-art approaches for finding maximum cliques in interdicted graphs.
- We also derive a battery of heuristic algorithms, to obtain high-quality incumbent solutions.
- All the above ingredients are combined in a computational framework which starts by calculating $\ell_{\min}$ and reducing the size of the input graph, before entering a branch-and-cut phase (B&C) in which the efficient separation procedure and primal heuristics are employed.

In an extensive computational study, we demonstrate the effectiveness of all the components of our exact solution framework. The study is based on publicly available benchmark instances from various sources, including large-scale graphs representing real-world (social) networks with up to one hundred thousand vertices and three million edges. Most of these instances are solved to provable optimality within a short computing time, thus significantly outperforming the state-of-the-art generic bilevel mixed integer programming solver of Fischetti et al. (2017).

Our new approach is a powerful tool for analyzing and assessing the graph resilience against clique-interdiction attacks, i.e., the capacity of the graph of preserving a large clique (and also its cohesiveness). For different classes of input graphs, we analyze the evolution of the clique number as a function of the interdiction budget k . Our analysis reveals that social networks are “vulnerable” to clique-interdiction attacks as their clique number can be easily reduced by a large factor using a relatively low interdiction budget.

Finally, we point out that our exact solution framework can be extended to a more generic interdiction problem in which the follower solves a *relaxed* clique problem with hereditary property, such as s -plex, s -bundle, or s -defective clique (see Pattillo et al. (2013) for a survey on clique relaxations).

The remainder of the paper is organized as follows: Section 2 concerns the literature review and Section 3 provides the bilevel integer programming formulation and complexity results. In Section 4, we provide theory and methodology behind our combinatorial lower bound and the associated reduction procedure. The single-level reformulation along with the polyhedral study is given in Section 5. The separation procedure is described in Section 6. Computational results are provided in Section 7, and concluding remarks are given in Section 8.

2 Related Literature

To the best of our knowledge, and despite its relevance for analyzing the cohesiveness of networks, the CIP is a new problem that has not been studied in the previous literature. There are only three problems directly related to the CIP that we are aware of. The first one is the *Minimum Vertex Blocker Clique Problem*, in which one searches for a subset of vertices of minimum cardinality to be removed from a graph G , so that the maximum (weighted) clique in the

remaining graph is bounded from above by a given integer $r \geq 1$. This problem has been studied in [Mahdavi Pajouh et al. \(2014\)](#), where an exact algorithm based on row-generation has been proposed. In a computational study conducted on randomly generated graphs, the authors report optimal solution values for sparse graphs with 100 and 200 vertices, whereas most of the instances with density 0.5 or above remain unsolved.

The second related problem is the *Edge Interdiction Clique Problem* which has been introduced in [Tang et al. \(2016\)](#) and used as a case study for a generic exact approach for (the more general class of) interdiction problems. In the edge interdiction clique problem, the leader interdicts edges instead of vertices, and therefore the majority of results derived in this article are not directly applicable to this problem. Nevertheless, it is worth mentioning that the largest graphs considered in the computational study of [Tang et al. \(2016\)](#) contain 15 vertices, and most of them remained unsolved within an hour of computing time. Optimal solutions for these instances have recently been provided by the state-of-the-art exact solver for bilevel mixed integer programs of [Fischetti et al. \(2017\)](#).

We also mention the recent work of [Bazgan et al. \(2011\)](#), where the authors introduce the problem of finding the *most vital vertices with respect to the maximum independent set*. This problem is closely related to CIP, as finding a maximum independent set on a graph is equivalent to finding a maximum clique on the complement graph. The authors of [Bazgan et al. \(2011\)](#) provide some complexity results by showing that their problem is polynomial-time solvable on unweighted bipartite graphs, on cographs and graphs with bounded treewidth.

Finally, CIP also belongs to a larger family of Interdiction Games under Monotonicity, which has been recently addressed in [Fischetti et al. \(2018\)](#). In this setting, the leader has a limited interdiction budget for deleting a subset of items, whereas the follower solves a maximization problem using the remaining items. The follower's subproblem is assumed to satisfy a monotonicity property, which is then exploited for deriving a single-level integer linear programming reformulation. The authors also study several strengthening inequalities which are of particular relevance for knapsack interdiction games. A computational study conducted on (multidimensional) knapsack interdiction games shows that this new approach significantly outperforms the previous state-of-the-art algorithm from ([Caprara et al., 2016](#)).

3 Bilevel formulation and problem complexity

In this section we first introduce the notation used in this article, we then provide a Bilevel Integer Linear Programming (ILP) formulation, after which we study the problem complexity.

3.1 Notation and definitions

Let $G = (V, E)$ be a simple undirected graph with $|V| = n$ and $|E| = m$. The complement of G is denoted $\overline{G} = (V, \overline{E})$, so $\overline{E} = \{uv : uv \notin E\}$. Given a vertex set S , the subgraph of G induced by S is denoted by $G[S]$. We say that u and v are *neighbours* if there is an edge $uv \in E$. Neighbors of a vertex v are denoted by $N(v)$. A subset $S \subseteq V$ of vertices is a *clique* of G , if any

two vertices of S are neighbours, and it is a *stable set* of G if it is a clique in $\overline{G}$. The cardinality of the largest clique of G is denoted by $\omega(G)$, and the cardinality of the largest stable set by $\alpha(G)$. The *maximum clique problem* (MCP) in G is to find a clique of maximum size.

Given a vertex $v \in V$, the *clique number of v* , denoted by $\omega_G(v)$ in the following, is the size of the largest clique v is contained in. The κ -*core* of a graph G is a maximal subgraph in which all vertices have degree at least κ . The *coreness-number* of a vertex v , denoted by $\text{coreness}(v)$ in the following, is equal to κ if v belongs to a κ -core but not to any $(\kappa + 1)$ -core. Obviously, the following inequality holds:

$$\omega_G(v) \leq \text{coreness}(v) + 1 \leq |N(v)| + 1 \quad v \in V. \quad (1)$$

For sparse graphs, the coreness-number of a vertex is often used as a rough approximation of its clique number, as it can be computed in $O(|E|)$ time for all vertices in G (for further details, see Batagelj and Zaversnik, 2011; San Segundo et al., 2016a). The maximum core number of a graph, denoted by $\text{core}(G)$ in the following, is the maximum of the core numbers of the vertices of G . So, we have

$$\omega(G) \leq \text{core}(G) + 1 \leq \max_{v \in V} \deg(v) + 1. \quad (2)$$

A *coloring* C of G is a partition of V into p non-empty stable sets: $C = \{V_1, \dots, V_p\}$, where all vertices belonging to V_i are colored with the same color i ($i = 1, \dots, p$). A graph property is called *hereditary on vertex induced subgraphs* if for any $S \subseteq V$ we have that if the property holds on $G[S]$, then it also holds on $G[S']$, for any $S' \subset S, S' \neq \emptyset$.

Let $\tilde{S}$ be the maximal stable set in G , i.e., $\alpha(G) = |\tilde{S}|$, hence $\omega(G) \leq |V| - \alpha(G)$. For a given interdiction budget k such that $k \geq |V| - \alpha(G)$ (i.e., there is enough budget to interdict at least all the vertices except those forming the maximum stable set), the optimal CIP solution value is at most one. Hence, we discard this trivial case and without loss of generality, we assume in the remainder of the paper that $k < |V| - \alpha(G)$.

3.2 Bilevel ILP formulation

The following binary decision variables are needed to model the problem as a bilevel integer linear program:

$$\begin{aligned} w_u &= \begin{cases} 1, & \text{if vertex } u \text{ is interdicted by the leader,} \\ 0, & \text{otherwise} \end{cases} & u \in V \\ x_u &= \begin{cases} 1, & \text{if vertex } u \text{ is used in the maximum clique of the follower,} \\ 0, & \text{otherwise} \end{cases} & u \in V \end{aligned}$$

Let $\mathcal{W}$ be the set of all feasible interdiction policies of the leader, i.e.:

$$\mathcal{W} = \left\{ w \in \{0, 1\}^n : \sum_{u \in V} w_u \leq k \right\}. \quad (3)$$

Similarly, let $\mathcal{K}$ represent the set of incidence vectors of all cliques in the graph G , i.e.:

$$\mathcal{K} = \{x \in \{0, 1\}^n : x_u + x_v \leq 1, uv \in \overline{E}\}, \quad (4)$$

where the constraints $x_u + x_v \leq 1$ ensure that two vertices cannot be part of a clique if there is no edge connecting them. With a slight abuse of notation, we will use both notations $K \in \mathcal{K}$ and $x \in \mathcal{K}$, where x is the incident vector of K . Given an interdiction strategy $w^* \in \mathcal{W}$, let V_{w^*} be the associated set of interdicted (deleted) vertices.

The CIP can be formulated as follows:

$$\min_{w \in \mathcal{W}} \max_{K \in \mathcal{K}} \left\{ |K| - \sum_{u \in K} w_u \right\}. \quad (5)$$

In the objective function of (5) we express the min-max nature of the problem: the leader controls (with the variables w) the outer minimization problem by interdicting at most k vertices of the graph, while the inner problem consists of calculating the clique number in the graph induced after the removal of the interdicted vertices. We call this inner problem the *follower's subproblem*). Observe that for the follower, the size of each clique K in G reduces by the number of interdicted vertices from K , which follows from the hereditary property of the clique (i.e., every vertex-induced subgraph of K is a clique itself). Therefore, the value $|K| - \sum_{u \in K} w_u$ denotes the size of the clique K after applying the interdiction strategy defined by $w \in \mathcal{W}$. Hence, the formulation (5) states that the leader chooses a set of vertices to interdict, so that among all possible cliques $K \in \mathcal{K}$, the size of the maximum remaining clique is the smallest possible.

The CIP can be equivalently stated as:

$$\min_{w \in \mathcal{W}} \max_{x \in \mathcal{K}} \left\{ \sum_{u \in V} (1 - w_u) x_u \right\} \quad (6)$$

Indeed, we can rewrite the objective function of the follower in (6) as $\sum_{u \in V} (1 - w_u) x_u = |K| - \sum_{u \in K} w_u x_u$. The latter sum corresponds to the objective function in (5) if and only if $w_u x_u = w_u$ if $u \in K$ and $w_u x_u = 0$, otherwise. The latter is always true, as the vector x encodes the clique K , and hence $x_u = 1$ if $u \in K$, and $x_u = 0$, otherwise. Finally, the CIP can also be reformulated as the following bilevel integer linear program:

$$\min_{w \in \mathcal{W}} \max_{x \in \mathcal{K}} \left\{ \sum_{u \in V} x_u : x_u \leq 1 - w_u, u \in V \right\}. \quad (7)$$

Indeed, given an interdiction strategy $w^* \in \mathcal{W}$, the optimal solution of the follower's subproblem (5) and the optimal solution of the follower's subproblem (7) are the same. Inequalities $x_u \leq 1 - w_u$ ($u \in V$) are the linking interdiction constraints, making sure that the follower cannot chose a vertex u if it has been interdicted by the leader. They ensure that for a given vector w^* , the follower searches for the maximum clique in the support graph in which the vertices v such that $w_v^* = 1$ have been removed. The bilevel ILP formulation (7) can be solved by using a Benders-like decomposition approach (cf. Section 5).

3.3 Problem complexity

In the following, we address the complexity of the decision version of CIP (denoted by d-CIP), stated as “*Is there an interdiction strategy such that the maximum clique in the interdicted graph is not greater than some given bound ℓ ?*”. Obviously, d-CIP is most probably not in NP for the following reason: Given a subset of k vertices to remove and a value of ℓ , to test whether the resulting graph does not contain a clique of size ℓ requires answering the decision problem of the maximum clique problem, which is NP-complete. [Rutenberg \(1991, 1994\)](#) proved a stronger result, namely that the d-CIP is complete for the complexity class Σ_2^P (see [Garey and Johnson \(1979\)](#), Chapter 7, or [Papadimitriou \(1994\)](#), Chapter 17 for further details on Σ_2^P). A slightly stronger complexity result is given in [Ko and Lin \(1995\)](#).

The CIP is solvable in polynomial time if G is a bipartite graph, or if it is a cograph, or if its complement graph $\bar{G}$ has a bounded treewidth. This result follows from the related study on the most vital vertices with respect to the maximum independent set given in [Bazgan et al. \(2011\)](#).

Proposition 3.1. *Assume that the graph $G = (K_1, \dots, K_p)$ is a union of vertex-disjoint cliques K_i ($1 \leq i \leq p$) which are given as part of the input and such that $|K_1| \geq \dots \geq |K_p|$ holds. The optimal solution value of the CIP can be found in $O(p)$ time, and an optimal solution can be calculated in $O(p + k)$ time.*

Proof. Let $\mathcal{Q}_q = (K_1, \dots, K_q)$ be the subgraph induced by the first q cliques of G , $1 \leq q \leq p$. Let $k(\mathcal{Q}_q)$ denote the size of an optimal interdiction strategy necessary to reduce the size of all cliques in $\mathcal{Q}_q$ to $|K_q| - 1$. One easily verifies that this number can be computed as:

$$k(\mathcal{Q}_q) = q + \sum_{i=1}^{q-1} i \cdot (|K_i| - |K_{i+1}|).$$

In the following we must consider two cases:

$k(\mathcal{Q}_p) > k$: In order to compute the optimal solution value, it is then sufficient to find the value q^* and the associated subgraph $\mathcal{Q}_{q^*}$ such that $k(\mathcal{Q}_{q^*}) > k$ and $k(\mathcal{Q}_{q^*-1}) \leq k$. In that case, the optimal solution value is obtained as:

$$\text{OPT}(G, k) = \max \left\{ |K_{q^*}|, |K_{q^*-1}| - 1 - \left\lfloor \frac{k - k(\mathcal{Q}_{q^*-1})}{q^* - 1} \right\rfloor \right\}. \quad (8)$$

By definition, $k(\mathcal{Q}_{q^*-1})$ is the budget necessary to interdict all cliques in $\mathcal{Q}_{q^*-1}$ of size $|K_{q^*-1}|$ or more, so that the largest clique will be of size $|K_{q^*-1}| - 1$. Then, the value $\left\lfloor \frac{k - k(\mathcal{Q}_{q^*-1})}{q^* - 1} \right\rfloor$ provides the number of units by which the size of the largest clique in $\mathcal{Q}_{q^*-1}$ can be further reduced using the remaining budget of $k - k(\mathcal{Q}_{q^*-1})$. Finally, if $|K_{q^*}| > |K_{q^*-1}| - 1 - \left\lfloor \frac{k - k(\mathcal{Q}_{q^*-1})}{q^* - 1} \right\rfloor$, the optimal solution value will be $|K_{q^*}|$. Finding the value of q^* can be done in $O(p)$ iterations, and each iteration has a constant time complexity, given the clique sizes.

$k(\mathcal{Q}_p) \leq k$: Using the same arguments, we can deduce

$$\text{OPT}(G, k) = |K_p| - 1 - \left\lfloor \frac{k - k(\mathcal{Q}_p)}{p} \right\rfloor. \quad (9)$$

To obtain an optimal solution, one has to (randomly) select $|K_i| - \text{OPT}(G, k)$ vertices from each of the cliques K_i ($1 \leq i \leq q^*$). This can be done in $O(k + p)$ time. $\square$

In order to see how to compute $\text{OPT}(G, k)$ using the formula (8), consider a graph G composed by four cliques ($p = 4$) of size 10, 8, 5 and 5. Then, $k(\mathcal{Q}_p) = 12$ and this corresponds to the budget k necessary to interdict all cliques of size 5 or more. Consider now a budget of $k = 7$, so we have $k(\mathcal{Q}_2) = 4$ and $k(\mathcal{Q}_3) = 11$, hence $q^* = 3$. The value of $\text{OPT}(G, 7)$ is given as $\max\{5, 6\}$, where 6 represents the size of the maximum clique after interdicting 7 vertices in the first two largest cliques. Consider now a budget of $k = 11$, in this case $q^* = 4$. The second term in formula (8) provides us the value of 4, as the size of the largest clique in $\mathcal{Q}_3$ after interdicting 11 vertices from the three largest cliques, whereas the size of the fourth clique provides the optimal solution value of 5 (as there is not sufficient budget to reduce its size).

Proposition 3.1 provides the optimal CIP solution value for the graphs composed by unions of cliques, but it is particularly important for deriving a tight globally valid lower bound in the most general case, without imposing any restrictions on G (cf. Section 4).

4 Combinatorial bounds, tighter gaps and preprocessing

We propose combinatorial algorithms for calculating tight global lower and upper bounds. Besides being useful for exact branch-and-bound-based approaches, we also demonstrate how these tight bounds help in reducing the size of the input graph. Let in the following $\ell_{\min}$, $\ell_{\max}$ denote the global lower and upper bound on the solution value, and let ℓ_{opt} be the optimal solution value (i.e., the size of the maximum clique after applying an optimal interdiction strategy).

4.1 Computing the global lower bound $\ell_{\min}$

In Section 3, we have shown that if graph G is composed by union of disjoint cliques, the problem is polynomial-time solvable. We now exploit this result to derive a combinatorial lower bound, which is obtained by removing edges from G . The following result allows us to compute a global lower bound on the CIP value.

Proposition 4.1. *Given a subgraph $G' = (V, E')$ with $E' \subset E$, an optimal CIP solution on G' provides a valid lower bound for the optimal CIP solution on G .*

Proof. Observe that $\omega(G') \leq \omega(G)$. Let $w \in \mathcal{W}$ be a feasible interdiction strategy for G and let V_w be the associated subset of interdicted vertices. The optimal CIP value on G is equal to

$$\min_{w \in \mathcal{W}} \omega(G[V \setminus V_w]).$$

Let $\bar{w} \in \mathcal{W}$ be an optimal interdiction strategy on G . Then we have

$$\omega(G'[V \setminus V_{\bar{w}}]) \leq \omega(G[V \setminus V_{\bar{w}}]) = \min_{w \in \mathcal{W}} \omega(G[V \setminus V_w]).$$

Since the most left expression corresponds to a feasible CIP solution on G' , and an optimal one is obtained by minimizing over all $w \in \mathcal{W}$, the result follows immediately. $\square$

The result of Proposition 4.1 is rather counter-intuitive, as it states that by reducing the input graph, instead of obtaining a valid upper bound for a minimization problem, we obtain a valid lower bound. The key issue here is that we are in fact not reducing the feasibility space of the leader (as the set of vertices in G' remains the same as in G), but only the feasibility space of the follower (which is reduced from all cliques induced by E to all cliques induced by E'). Observe, furthermore, that in terms of the problem complexity, the result of Proposition 4.1 does not help us much in solving the problem, since we still have to solve the same CIP problem, only on a smaller graph. However, this result can be particularly useful for special classes of graphs on which solving the CIP on G' is easier than on G . This is exploited by the following result, where the proof for correctness of formula (10) follows from the proof of Proposition 3.1.

Corollary 4.2. *Given a set $\mathcal{Q}_{p+1} = (K_1, \dots, K_{p+1})$ of vertex-disjoint cliques of G , such that $|K_1| \geq \dots \geq |K_{p+1}|$, a valid lower bound $\ell_{\min}$ for the CIP can be obtained by computing*

$$\ell_{\min} = \begin{cases} \max \left\{ |K_{p+1}|, |K_p| - 1 - \left\lfloor \frac{k - k(\mathcal{Q}_p)}{p} \right\rfloor \right\}, & \text{if } k < k(\mathcal{Q}_{p+1}) \\ |K_{p+1}| - 1 - \left\lfloor \frac{k - k(\mathcal{Q}_{p+1})}{p+1} \right\rfloor, & \text{otherwise} \end{cases} \quad (10)$$

The quality of this lower bound depends on the number of cliques $p + 1$ and their sizes (see Section 7 for the discussion on the empirical quality of the bounds and further implementation details).

4.2 Reducing the input graph

We start by describing another important solution property that is exploited in our study. Given a clique K and an interdiction strategy $w^* \in \mathcal{W}$, we say that K is *covered* by V_{w^*} if and only if at least one vertex from K is interdicted by w^* , i.e., iff $K \cap V_{w^*} \neq \emptyset$.

Property 4.3. *If there exists a feasible interdiction strategy $w^* \in \mathcal{W}$, such that all cliques of size $\ell_{\max} + 1$ in G are covered by V_{w^*} and there exists at least one clique K^* in G , $|K^*| = \ell_{\max}$ which is not covered by V_{w^*} , then $\ell_{\max}$ is a valid (non-trivial) upper bound on the CIP.*

The latter property suggests another way of seeing the CIP: find a feasible interdiction strategy that minimizes the value of $\ell_{\max}$ while making sure all cliques of size $\ell_{\max} + 1$ are covered.

We say that an interdiction strategy $w \in \mathcal{W}$ is *minimal*, if for the associated set of interdicted vertices V_w , we have:

$$\omega(G[V \setminus V_w]) < \omega(G[(V \setminus V_w) \cup \{v\}]), \quad \forall v \in V_w.$$

The following result identifies redundant vertices in the input graph G .

Proposition 4.4. *Let v be an arbitrary vertex from V . If $\omega_G(v) \leq \ell_{\text{opt}}$, then v cannot be part of a minimal optimal interdiction strategy.*

Proof. By contradiction, assume that there exists a minimal optimal interdiction strategy $w^* \in \mathcal{W}$ such that $w_v^* = 1$. We now construct a new feasible interdiction strategy $\bar{w} \in \mathcal{W}$ as follows: $\bar{w}_u := w_u^*$ for all $u \in V, u \neq v$ and $\bar{w}_v := 0$. It remains to show that

$$\omega(G[V \setminus V_{\bar{w}}]) = \omega(G[V \setminus V_{w^*}]) = \ell_{\text{opt}}.$$

Clearly, $\omega(G[V \setminus V_{\bar{w}}]) \geq \ell_{\text{opt}}$, since $\bar{w}$ is a feasible interdiction strategy. By Property 4.3, an optimal solution V_{w^*} must cover all cliques of size $\geq \ell_{\text{opt}} + 1$. Since the largest cliques containing v are of size $\leq \ell_{\text{opt}}$, then also the set $V_{\bar{w}} = V_{w^*} \setminus \{v\}$ covers all the cliques of size $\geq \ell_{\text{opt}} + 1$, which implies that $\omega(G[V \setminus V_{\bar{w}}]) \leq \ell_{\text{opt}}$ and this concludes the proof. $\square$

Hence, by focusing on the minimal interdiction policies, which can be done without loss of generality, one can preprocess the graph G and remove redundant vertices from it. To properly exploit Proposition 4.4, instead of using the (unknown) value of ℓ_{opt} for removing the redundant vertices, one can employ a tight lower bound $\ell_{\min}$. The following result gives a connection between the optimal solution value and the solution value found on the preprocessed graph from which redundant vertices are removed.

Proposition 4.5. *Let $V_{\text{prep}} = \{v \in V : \omega_G(v) \leq \ell_{\min}\}$ be the set of vertices v satisfying the (weakened) property of Proposition 4.4. Let $\tilde{V} = V \setminus V_{\text{prep}}$ and $\tilde{G} = G[\tilde{V}]$ and let $\tilde{\ell}_{\text{opt}}$ denote the optimal CIP solution value on $\tilde{G}$. Then*

$$\ell_{\text{opt}} = \max\{\tilde{\ell}_{\text{opt}}, \ell_{\min}\}.$$

Furthermore, an optimal interdiction strategy $\tilde{w}$ on $\tilde{G}$, is also optimal for G .

Proof. Let $\tilde{w}$ be an optimal interdiction strategy found on $\tilde{G}$. To show that $\tilde{w}$ is also optimal for G , we distinguish between the following two cases:

1. $\omega(G[\tilde{V} \setminus V_{\tilde{w}}]) < \ell_{\min}$: In that case, $\tilde{w}$ is a feasible interdiction strategy on G , with the follower's optimal response on the remaining graph with value $\ell_{\min}$. This is due to the facts that: (i) all vertices from V_{prep} cannot appear in a clique of size larger than $\ell_{\min}$, (ii) $\ell_{\min}$ is a valid lower bound and thus there exist at least one clique of size $\ell_{\min}$ in the graph $G[V \setminus V_{\tilde{w}}]$. Hence, $\ell_{\min}$ is equal to the optimal solution value ℓ_{opt} .
2. $\omega(G[\tilde{V} \setminus V_{\tilde{w}}]) \geq \ell_{\min}$: In that case, $\tilde{\ell}_{\text{opt}} = \ell_{\text{opt}}$, due to the fact that interdicting any of the vertices from V_{prep} will not result in a better interdiction strategy, because the largest clique that can be covered by interdicting vertices from V_{prep} is of size $\ell_{\min}$. This concludes the proof.

□

Observe that the latter result allows us not only to fix the binary decision variables w_v to zero, but also to modify the input graph G by removing its vertices, resulting into a smaller input graph for solving the follower's subproblem. This reduction preserves optimality, since the removal of a vertex $v \in V_{\text{prep}}$ only reduces the size of maximal cliques K such that $|K| \leq \ell_{\min}$, which are never going to constitute an optimal follower's response to an optimal interdiction strategy.

In a standard implementation, one would start with an arbitrary vertex, and solve the MCP by fixing this vertex to one. If the size of the obtained clique K is smaller than $\ell_{\min}$, vertex v is removed from G and the process is repeated for the remaining vertices. This procedure can be time consuming, as the maximum clique algorithm has to be called per each vertex. To overcome this drawback, the degree of a vertex can be used instead, as a trivial upper bound on the clique number of a vertex v . However, according to inequality (1), a much more accurate bound is the coreness-number of a vertex. Hence, in our implementation, we pre-calculate the coreness-number for all vertices (in the initialization phase), and then remove all vertices v such that $\text{coreness}(v) \leq \ell_{\min}$.

4.3 Computing the global upper bound $\ell_{\max}$

We have implemented several construction heuristics in order to calculate a tight upper bound $\ell_{\max}$ and to create a pool of initial feasible solutions that are later given to the MIP solver. In all heuristics, if a vertex has been fixed to zero (e.g., removed by the preprocessing described above), it is not considered as a candidate for being interdicted. Our heuristics work in a greedy fashion, based on four different criteria: vertices are interdicted one-by-one, until the interdiction budget is exhausted. The chosen criteria are as follows:

- Vertex-degree: At the beginning, the vertices are sorted in non-increasing order according to their degrees. We start by interdicting the vertices with the highest degree first, and we stop once the interdiction budget is exhausted.
- Updated vertex-degree: The major difference to the “vertex-degree” heuristic is in the fact that now we recompute the vertex degrees, each time a vertex is interdicted.

- **Vertex-coreness-number:** at the beginning, the vertices are sorted in non-decreasing order according to their coreness number. Intuition behind this approach is that a vertex with a high coreness value is likely to belong to large cliques and shall be interdicted first. As mentioned above, $\text{coreness}(v) + 1$ is a rough upper bound on the size $\omega_G(v)$ of the maximum clique containing this vertex.
- **Vertex-color-number:** This heuristic exploits the well-known result that the size of any feasible coloring gives an upper bound on the clique number of a graph (Balas and Yu, 1986). We first apply the greedy sequential coloring heuristic based on independent sets, where color classes are obtained incrementally using bitmasks, see (San Segundo et al., 2011, 2013). This procedure runs in $O(|V|^2)$ in the worst case and returns a feasible coloring. We then assign a label (corresponding to the color number) to each vertex. In this sequential greedy coloring heuristic, the higher the color associated to a vertex, the higher are the chances that this vertex belongs to a large clique. Therefore, we interdict the vertices in the non-increasing order with respect to their color numbers. Notice that the reordering of vertices, based on their color number, once they have been interdicted, will not influence the color number. So, there is no need to reorder the vertices after partially interdicting them, as long as they are interdicted starting with the highest color number first. As it will be shown in our computational study, vertex-color-number provides a very robust measure that delivers excellent heuristic solutions for sparse social networks.

5 Single-level ILP reformulation

In this section we provide an ILP formulation of the problem in the natural space of leader decision variables w . In addition, we provide a facial study of the underlying polytope, discussing under which conditions the proposed inequalities are facet defining.

The following is a valid ILP formulation for CIP:

$$\min \quad \theta \tag{11}$$

$$\theta + \sum_{u \in K} w_u \geq |K| \quad K \in \mathcal{K} \tag{12}$$

$$\sum_{u \in V} w_u \leq k \tag{13}$$

$$w_u \in \{0, 1\} \quad u \in V. \tag{14}$$

To see that the model is valid, observe that for every feasible interdiction strategy $\bar{w} \in \mathcal{W}$, the follower's problem boils down to $\max_{x \in \mathcal{K}} \sum_{u \in V} x_u(1 - \bar{w}_u)$. Hence, the problem can be restated so that the set of feasible solutions of the follower does not depend on the actions of the leader anymore. Consequently, one can enumerate all cliques in G and optimize over the set $\mathcal{K}$. This is

why the problem can be equivalently restated as

$$\min_{w \in \mathcal{W}} \left\{ \theta : \theta \geq \sum_{u \in V} \bar{x}_u (1 - w_u), \bar{x} \in \mathcal{K} \right\},$$

where $\bar{x}$ represents an arbitrary incidence vector of a clique in G .

This single-level ILP formulation contains an exponential number of constraints of type (12) that we will refer to as *Clique Interdiction* (CI) cuts. These constraints are NP-hard to separate: for each vector $\bar{w}$ and the associated $\bar{\theta}$ given by the current solution of the formulation (12) and (13), checking if there exists a violated interdiction cut requires finding a maximum weighted clique on G with vertex-weights $c_u = 1 - \bar{w}_u$ for all $u \in V$.

In a branch-and-cut algorithm applied to the above ILP formulation, it is sufficient to separate integer infeasible points only (fractional points being cut off using standard branching and general cutting plane mechanisms embedded in modern MIP solvers). Whenever $\bar{w}$ is integer, the separation problem consists in solving the MCP in the support graph $G[V \setminus V_{\bar{w}}]$. Let $\bar{K}$ be the maximum clique in $G[V \setminus V_{\bar{w}}]$: if $|\bar{K}| > \bar{\theta}$ then a violated CI cut associated to $\bar{K}$ is found and added to the model. This separation procedure can be a potential bottleneck for using a branch-and-cut procedure, unless an efficient clique solver is used for the separation of CI cuts. We have therefore implemented a tailored separation algorithm based on recent state-of-the-art approaches for the MCP (see Section 6 for further details).

5.1 Valid inequalities and facial study

In the following, we study the polytope of the single-level CIP formulation (11)-(14). We provide necessary and sufficient conditions under which the clique interdiction cuts (12) are facet defining and we discuss heuristic lifting procedures designed to strengthen these inequalities.

Given the graph G and the interdiction budget k , let $\mathcal{P}(G, k)$ denote the convex hull of feasible solutions of the CIP formulation (11)-(14), that is,

$$\mathcal{P}(G, k) = \text{conv} \left\{ w \in \{0, 1\}^{|V|}, \theta \geq 0 : \theta + \sum_{u \in K} w_u \geq |K|, K \in \mathcal{K}, \sum_{u \in V} w_u \leq k \right\}.$$

Let (V', q) denote a CIP solution where $\theta = q$ and the interdiction strategy is defined by V' .

Proposition 5.1. *The polytope $\mathcal{P}(G, k)$ is full dimensional.*

Proof. We will prove this result by contradiction. Suppose that $\mathcal{P}(G, k)$ is contained in a hyperplane defined by the linear equation $\sum_{u \in V} \alpha_u w_u + \beta \theta = \gamma$, where $\alpha \in \mathbb{R}^{|V|}$, $\beta \in \mathbb{R}$, $(\alpha, \beta) \neq 0$, and $\gamma \in \mathbb{R}$. We show that $\alpha = 0$, $\beta = 0$ and thus $\mathcal{P}(G, k)$ cannot be included in this hyperplane.

Let $v \in V$ be an arbitrary vertex of the graph G . The three solutions $(V_1 = \emptyset, |V|)$, $(V_v = \{v\}, |V| - 1)$ and $(V_v = \{v\}, |V|)$ are valid for $\mathcal{P}(G, k)$. We deduce that, $\alpha^{V_v} + \beta \cdot |V| =$

$\alpha^{V_v} + \beta \cdot (|V| - 1)$ and thus $\beta = 0$. We also deduce that, $\alpha^{V_1} + \beta \cdot |V| = \alpha^{V_v} + \beta \cdot |V|$, this implies $\alpha_v = 0$ for all $v \in V$. $\square$

Proposition 5.2. *Let $u \in V$.*

1. *The trivial inequality $w_u \leq 1$ defines a facet of $\mathcal{P}(G, k)$ if and only if $k \geq 2$.*
2. *The trivial inequality $w_u \geq 0$ defines a facet of $\mathcal{P}(G, k)$.*

Proof. 1. Let $u \in V$. If $k = 1$ then the inequality (13) dominates the inequality $w_u \leq 1$. For $k \geq 2$, consider the following $|V| + 1$ feasible solutions that belong to $\mathcal{P}(G, k)$ and satisfy the inequality $w_u \leq 1$ with equality: $(V_0 = \{u\}, |V|)$, $(V_1 = \{u\}, |V| - 1)$ and $(V_v = \{u, v\}, |V|)$ for all $v \in V \setminus \{u\}$. Clearly, these solutions are affinely independent, which concludes the proof.

2. Consider the following $|V| + 1$ feasible solutions from $\mathcal{P}(G, k)$ that satisfy $w_u \geq 0$ with equality: $(V_0 = \emptyset, |V|)$, $(V_v = \{v\}, |V|)$ for all $v \in V \setminus \{u\}$ and $(V_1 = \{v'\}, |V| - 1)$ for some $v' \neq u$. Clearly, these solutions are affinely independent, so $w_u \geq 0$ defines a facet. $\square$

Lemma 5.3. *Let $K \in \mathcal{K}$ be an arbitrary clique in G . The associated clique interdiction inequality (12) does not induce a facet if:*

1. $|K| \leq \ell_{\text{opt}}$, or
2. K is not maximal.

Proof. 1. Observe that, by definition, we have $\theta \geq \ell_{\text{opt}}$. Hence, for $|K| < \ell_{\text{opt}}$ the equality

$$\sum_{u \in K} w_u = |K| - \theta$$

has a negative right-hand side, i.e., there is no feasible solution satisfying this equation. On the other hand, for $|K| = \ell_{\text{opt}}$, the CI cut $\theta + \sum_{u \in K} w_u = |K|$ is dominated by $\theta \geq \ell_{\text{opt}}$.

2. Obvious. $\square$

Even though the value of ℓ_{opt} is not known in advance, the above result is useful for the separation of clique interdiction cuts. The result states that the more promising inequalities (in terms of improving the quality of lower bounds) are those with $|K| \geq \ell_{\text{min}}$, considering thereby the value of ℓ_{min} as tight as possible. This is also in line with our preprocessing procedure that removes all vertices from G whose clique number is not greater than ℓ_{min} . In addition, without loss of generality, in the remainder of this section we focus on maximal cliques only.

Lemma 5.4. *Let K be a maximal clique and $v \in K$. If*

$$\omega(G[V \setminus V']) \geq |K| - |V' \cap K| + 1 \quad \forall V' \subseteq V \text{ where } v \in V' \text{ and } |V'| \leq k, \quad (15)$$

then there exists $\alpha_v \leq 0$ such that the associated clique interdiction cut (12) can be down-lifted to

$$\theta + \sum_{u \in K \setminus \{v\}} w_u + \alpha_v w_v \geq |K|.$$

Proof. Let K be a maximal clique and let $v \in K$. The inequality $\theta + \sum_{u \in K} w_u \geq |K|$ is valid. Using a lifting procedure, see (Nemhauser and Wolsey, 1988, p. 261), the coefficient α_v of w_v can be calculated as

$$\alpha_v = |K| - \min \left\{ \theta + \sum_{u \in K \setminus \{v\}} w_u \mid (w, \theta) \in \mathcal{P}(G, k) \text{ and } w_v = 1 \right\}. \quad (16)$$

Recall that to any feasible interdiction strategy w , we can associate the set $V' \subset V$, $|V'| \leq k$ of vertices to be deleted from G . Hence the minimum value of the right hand side is equal to $\min_{V'} \{\omega(G[V \setminus V']) + |K \cap V'| - 1\}$ where the latter minimum is calculated over all feasible interdiction policies V' that contain v . According to our condition (15), the size of the maximum clique in the graph $G[V \setminus V']$ plus the number of interdicted vertices which are in K , including v , is greater or equal $|K| + 1$. Then, $\min\{\theta + \sum_{u \in K \setminus \{v\}} w_u \mid (w, \theta) \in \mathcal{P}(G, k) \text{ and } w_v = 1\} \geq |K|$ and therefore $\alpha_v \leq 0$. $\square$

Corollary 5.5. *Let $K \subset V$ be a clique. If there exists $v \in K$ satisfying (15) then the inequality (12) cannot define a facet.*

Finally, the following Proposition provides necessary and sufficient conditions under which the CI cuts are facet defining. This is the major theoretical result of this section, which allows to characterize the strength of the ILP formulation upon which our solution framework is built.

Proposition 5.6. *Let $K \in \mathcal{K}$ be a maximal clique. Inequality (12) associated with K defines a facet of $\mathcal{P}(G, k)$ if and only if*

- $|K| \geq \ell_{\text{opt}} + 1$
- for all $v \in K$, there exists a subset $V' \subseteq V$ such that $v \in V'$, $|V'| \leq k$ and $\omega(G[V \setminus V']) + |V' \cap K| \leq |K|$.

Proof. ($\Rightarrow$) See Lemma 5.3 and Corollary 5.5.

($\Leftarrow$) Let us denote by $\alpha'w + \beta'\theta \geq \gamma'$ inequality (12) associated with K . Let $\alpha w + \beta\theta \geq \gamma$ be a facet containing an equality (12) such that $\{(w, \theta) \in \mathcal{P}(G, k) : \alpha'w + \beta'\theta = \gamma'\} \subseteq \{(w, \theta) \in \mathcal{P}(G, k) : \alpha w + \beta\theta = \gamma\}$. We will show that $\alpha = \rho\alpha'$ and $\beta = \rho\beta'$ for some $\rho \in \mathbb{R}$.

Let us first consider a set V' of vertices which is an optimal interdiction strategy such that $V' \cap K \neq \emptyset$. Observe that such a set must exist, given that the optimal solution value is ℓ_{opt} and $|K| \geq \ell_{\text{opt}} + 1$, so at least one vertex from K has to be interdicted. Let $V'' = V' \setminus K$.

- We will first show that $\alpha_v = 0$ for all $v \in V''$. Let $v \in V''$. We consider the solution $(V_1 = V'', |K|)$. This solution is feasible since $\omega(G[V \setminus V']) = \ell_{\text{opt}} \leq |K|$. Consider a second solution $(V_2 = V'' \setminus \{v\}, |K|)$. This solution is also feasible because by interdicting one vertex less from V'' , the size of the maximum clique in $G[V \setminus (V'' \setminus \{v\})]$ is at most $\ell_{\text{opt}} + 1 \leq |K|$. These two solutions satisfy (12) with the equality. We deduce $\alpha^{V_1} + \beta \cdot |K| = \alpha^{V_2} + \beta \cdot |K|$. This implies $\alpha_v = 0$ for all $v \in V''$.
- We now show that $\alpha_v = 0$ for all $v \in (V \setminus K) \setminus V''$. Let $v \in (V \setminus K) \setminus V''$. Let us consider the solution $(V_3 = V_2 \cup \{v\}, |K|)$. This solution is valid since we interdict an additional vertex compared to V_2 , so $\omega(G[V \setminus (V_2 \cup \{v\})]) \leq \omega(G[V \setminus V_2]) \leq |K|$. Clearly this solution satisfies (12) with equality. We deduce $\alpha^{V_1} + \beta \cdot |K| = \alpha^{V_3} + \beta \cdot |K|$. This implies $\alpha_v = 0$ for all $v \in (V \setminus K) \setminus V''$.

Let $v_k \in K$. By the hypothesis there exists a set V_k of vertices which is a feasible interdiction strategy ($|V_k| \leq k$) such that $v_k \in V_k$ and $\omega(G[V \setminus V_k]) + |V_k \cap K| \leq |K|$. By the inequality (12) ($\theta + \sum_{u \in K} w_u \geq |K|$) we deduce that $\omega(G[V \setminus V_k]) + |V_k \cap K| = |K|$. Let $V'_k = V_k \setminus K$.

- We will show that $\beta = \alpha_{v_k}$, for the given $v_k \in K$. Consider two feasible solutions $(V_k, \omega_0 = \omega(G[V \setminus V_k]))$ and $(V_1 = V_k \setminus \{v_k\}, \omega_1 = \omega(G[V \setminus V_k]) + 1)$ – they both satisfy (12) with equality. This implies $\alpha^{V_k} + \beta \cdot \omega_0 = \alpha^{V_1} + \beta \cdot \omega_1$ and thus $\alpha_{v_k} = \beta$.
- Finally, we will show that $\alpha_{\tilde{v}} = \alpha_{v_k}$, for all $\tilde{v} \in K, \tilde{v} \neq v_k$. Let $\tilde{v} \in K$ and let $\tilde{V}$ be a feasible interdiction strategy such that $\tilde{v} \in \tilde{V}$, and $\omega(G[V \setminus \tilde{V}]) + |\tilde{V} \cap K| \leq |K|$. Let $\tilde{V}' = \tilde{V} \setminus K$ and $V'_k = V_k \setminus K$. We consider $(V_4 = V'_k \cup \{v_k\}, |K| - 1)$ and $(V_5 = \tilde{V}' \cup \{\tilde{v}\}, |K| - 1)$ two valid solutions. Indeed, in the graph $G[V \setminus V'_k]$ there remains the clique K and if we interdict one vertex of K plus the vertices of V'_k then the biggest clique has a size at most $|K| - 1$. The same argument holds for the solution $(V_5, |K| - 1)$. Hence we can deduce that these two solutions satisfy (12) with equality. This implies $\alpha^{V_4} + \beta \cdot (|K| - 1) = \alpha^{V_5} + \beta \cdot (|K| - 1)$. Since $\alpha_v = 0$ for all $v \in V \setminus K$, it follows that $\alpha_{\tilde{v}} = \alpha_{v_k}$, for all $\tilde{v}, v_k \in K$.

To finish the proof, we set $\beta = \rho$ and thus $\alpha = \rho\alpha'$ and $\beta = \rho\beta'$. $\square$

5.2 Heuristic lifting procedure

According to the proof of Lemma 5.4, down-lifting coefficients of a clique interdiction cut requires solving another optimization problem given by (16), which is again a CIP on a smaller instance. We can apply some heuristic ideas instead, by underestimating the left-hand-side and overestimating the right-hand-side of the condition (15).

Observe that the left-hand-side of this inequality can be calculated as:

$$\ell_{\text{opt}}(v) = \min\{\theta \mid (w, \theta) \in \mathcal{P}(G, k) \text{ and } w_v = 1\}$$

which is the value of an optimal CIP solution, assuming vertex v is interdicted. The value of $\ell_{\text{opt}}(v)$ can be underestimated by calculating $\ell_{\min}(v)$, which is the lower bound obtained using the formula provided in Corollary 4.2. However, a slight modification is needed, to ensure this bound is valid for the case vertex v is interdicted. To obtain the value of $\ell_{\min}(v)$, we first remove vertex v from G , and then we compute vertex-disjoint cliques $\mathcal{Q}_p$ on the fly, and stop as soon as we find p^* such that $k(\mathcal{Q}_{p^*+1}) > k - 1$ (as the vertex v already consumes one unit of the interdiction budget). Notice that the values $\ell_{\min}(v)$ need to be calculated only once in the initialization phase, and they remain valid through the whole branch-and-cut procedure.

The right-hand-side of the condition (15) is simply overestimated by $|K|$ (knowing that $v \in V' \cap K$). This results in the following heuristic condition for down-lifting the coefficients of the clique-interdiction cuts that can be performed in a computationally efficient way.

Proposition 5.7. *Let K be a maximal clique and $v \in K$. Let LB be the lower bound in the current node of the branch-and-bound tree. If $\max\{\ell_{\min}(v), \lceil LB \rceil - 1\} \geq |K|$ then the down-lifted clique-interdiction cut*

$$\theta + \sum_{u \in K, u \neq v} w_u \geq |K|$$

is valid.

Proof. The proof follows from the condition (15) which ensures that in that case α_v , the coefficient next to v in a valid clique-interdiction constraint is such that $\alpha_v \leq 0$. The left-hand-side of (15) is underestimated by $\max\{\ell_{\min}(v), \lceil LB \rceil - 1\}$. We have to subtract the value of 1 from LB , since we do not know if vertex v has been interdicted or not, when this bound is calculated. The right-hand-side of this condition is overestimated by $|K|$. $\square$

Observe that such down-lifted clique interdiction cuts are only locally valid, and the globally valid ones can be similarly obtained by considering a global lower bound LB_g instead.

6 Solving the separation problem by tailoring a state-of-the-art clique solver

The separation problem requires solving the MCP in a number of induced subgraphs $G[V \setminus V_w]$, where V_w is a feasible interdiction strategy. To solve each MCP exactly, we have designed a specific branch-and-bound (B&B) algorithm inspired by the ideas described in Li et al. (2017); San Segundo et al. (2016a), using a compact bitstring representation both for vertex sets and the adjacency matrix of the input graph. This B&B, denoted by IMCQ in the following, is a purely combinatorial approach that exploits tight lower and upper bounds for the clique number, along with efficient branching rules.

Initialization IMCQ starts with a preprocessing phase, where the vertices of the input graph are reordered, a feasible solution K_0 and a tight upper bound ub are calculated, and a branching candidate set B_0 is determined.

The vertex ordering is essential for the efficient use of cache memory for bitstring graph representations and for reducing the size of the search tree, see (San Segundo et al., 2011, 2013). We distinguish between two different categories of input graphs: large-scale sparse graphs and small or middle-size ($n < 5,000$) but dense graphs. In the remainder of the paper, these categories will be referred to as *sparse* and *non-sparse*. For sparse graphs, vertices are sorted according to non-decreasing coreness in a degeneracy ordering using the algorithm of Batagelj and Zaversnik (2011). For non-sparse graphs, vertices are sorted according to their degrees, following the approach from, e.g., San Segundo et al. (2016a).

In the case of sparse graphs, we use the fixed upper bound $ub = 1 + \text{core}(G)$ for all interdiction strategies, where $\text{core}(G)$ is the core number of G . In the case of non-sparse graphs, $ub = \min\{\omega(G), 1 + \text{core}(G[V \setminus V_w])\}$. For computing an initial solution K_0 , we use a simple greedy heuristic (see San Segundo et al. (2016a)).

The last initialization step is to determine an initial branching set of vertices $B_0 \subset V \setminus V_w$, which are the candidate vertices which may belong to a clique larger than $|K_0|$. B_0 is then determined by all the vertices $v \in V \setminus V_w$ such that $1 + \text{coreness}(v) > |K_0|$, where $\text{coreness}(v)$ is the maximum core number of any subgraph that contains v .

Branching We now describe the key ideas used to determine the candidates for branching, which are applied at each node of the B&B procedure. Given an input graph $G = (V, E)$ and an integer q (corresponding to a valid lower bound on the MCP on G), it is possible to partition V into two disjoint sets of vertices P and $B = V \setminus P$ such that $\omega(G[P]) \leq q$, so that an optimal MCP solution can be found by branching on the vertices from B only. Reducing the size of the set B is crucial for an efficient implementation of a B&B-based MCP solver, see e.g., (San Segundo and Tapia, 2014; San Segundo et al., 2015, 2016b; Li et al., 2017). Computing $\omega(G[P])$ can be computationally expensive, and for this reason, this value is often replaced by an upper bound.

For sparse graphs, this upper bound is provided by a feasible κ -coloring on G , see San Segundo et al. (2011, 2013). If κ is strictly greater than q , the branching set B is determined by the union of vertices in color classes with a color number higher than q ; otherwise no branching candidates exist and the algorithm backtracks. For the non-sparse graphs, we further reduce such obtained set B by an additional *infrachromatic* bounding function, see (San Segundo et al., 2015; Li et al., 2017). This function takes as input the feasible κ -coloring on G and reduces the upper bound κ by one unit each time a subset I of independent sets of the coloring is found such that it cannot contain a clique of size $|I|$. Our current implementation is inspired by the MaxSAT-based infrachromatic function described in (Li et al., 2017), and tailored for the bitstring representation of G .

The infrachromatic bounding function is applied incrementally at every search node as follows. The P and B sets are initially determined by the greedy κ -coloring. Then, for every vertex $v \in B$, the function is called to determine whether or not $\omega(G[P \cup \{v\}]) \leq q$. Every time the condition holds, v is added to P and the enlarged set $P' = P \cup \{v\}$ is taken as reference in

future iterations. After all the vertices in B have been processed, IMCQ will branch on the (few) vertices remaining in B .

At the beginning, $G = G[V \setminus V_w]$, and each time a vertex $v \in B$ is selected for branching (i.e., for inclusion into a clique under construction), the graph G is reduced by removing the non-neighbor vertices of v . The value of q is then determined as the difference between the size of the incumbent clique, and the size of the clique under construction.

7 Computational results

The purpose of this computational study is threefold: (1) to compare the exact solution framework for CIP presented in this paper with the state-of-the-art method available for general bilevel and interdiction problems; (2) to assess the practical applicability and the limits of our framework on large-scale social networks, and (3) to use this framework to analyze the resilience of (social) networks with respect to vertex-interdiction attacks, i.e., the decrease of the size of the maximum clique as a function of the incremental interdiction budget levels k .

In the following we first summarize our exact solution framework, followed by the explanation of benchmark instances and a detailed computational study. All the experiments have been performed on a computer with a 3.40 Ghz 8-core Intel Core i7-3770 processor and 16Gb RAM, running a 64-bit Linux operating system. The source codes were compiled with `gcc 4.8.4` and `-O3` optimizations. We used `CPLEX 12.7.0` and the `CALLABLE LIBRARIES` framework to implement our branch-and-cut algorithm. `CPLEX` was run in single-threaded mode and all `CPLEX` parameters were set to their default values.

7.1 The exact solution framework CLIQUE-INTER

Our exact solution framework is based on the CIP formulation (11)-(14) of Section 5. It is a branch-and-cut procedure combined with a preprocessing and efficient combinatorial algorithms for the computation of lower and upper bounds, and for the separation procedure. The framework is called CLIQUE-INTER in the remainder of the paper. These are its main components:

- (i) An effective separation procedure of the CI cuts (12): To this end we apply the specialized combinatorial B&B algorithm (IMCQ) described in Section 6. IMCQ is an enhancement of one of the state-of-the-art MCP algorithms of San Segundo et al. (2016a) capable of effectively solving the MCP once the vertices of an interdiction strategy have been removed from the graph G . In addition, sharper cuts are obtained by *maximal* cliques (see Lemma 5.3); for this reason, before adding each CI cut, we make the associated clique maximal, i.e., potentially enlarging it with vertices that have been interdicted.
- (ii) Tight CIP upper and lower bounds (ℓ_{min} and ℓ_{max}): In order to initialize the lower bound value of the variable θ of formulation (11)-(14), we used the global lower bound ℓ_{min} presented in Section 4.1. We create the subset $\mathcal{Q}_{p+1}$ required by Corollary 4.2 by iteratively extracting vertex-disjoint maximum cliques using IMCQ from G until we find the first p^* such that $k(\mathcal{Q}_{p^*+1}) > k$. In this way we minimize the number of times the

NP-hard maximum clique problem is solved. On the other hand, by focusing on maximum (rather than random cliques), we preserve the quality of the bound $\ell_{\min}$ in a greedy fashion. By construction, the generated cliques are sorted in non-increasing order by their size, as required by the formula (10).

The cliques from the set $\mathcal{Q}_{p^*+1}$ are also used to create an initial pool of CI cuts. In order to define a high-quality feasible CIP solution of value $\ell_{\max}$ to initialize formulation (11)-(14), we apply a battery of sequential greedy heuristics presented in Section 4.3. If it turns out that $\ell_{\min} = \ell_{\max}$, the instance is solved to optimality, before even starting the B&C procedure.

- (iii) The graph reduction: For large-scale real-world graphs, formulation (11)-(14) becomes easily intractable, unless the input graph can be safely reduced to a smaller one. The reduction technique presented in Section 4.2, which exploits the tight lower bound $\ell_{\min}$ and preserves the optimality of the solution, is applied before calculating an initial upper bound $\ell_{\max}$ and entering the B&C phase.

As later shown in this computational section, all three components are crucial and necessary to guarantee an efficient computational performance of **CLIQUE-INTER**.

7.2 Test-bed of instances

In order to assess the performance of the newly proposed exact algorithm **CLIQUE-INTER**, we consider three sets of benchmark instances, called *Set A*, *Set B* and *Set C*:

Set A – Random graphs We created a set of 55 Erdős-Rényi random $G(n, p)$ graphs of different sizes ($n = |V| \in \{50, 75, 100, 125, 150\}$) and edge densities ($p \in \{0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 0.95, 0.98\}$). We set k to four different percentages of $|V|$, i.e., $k \in \{\lceil 0.05 \cdot |V| \rceil, \lceil 0.1 \cdot |V| \rceil, \lceil 0.2 \cdot |V| \rceil, \lceil 0.4 \cdot |V| \rceil\}$. In this way, we created the Set A of 220 instances.

Set B – Synthetic graphs This set contains 16 well known synthetic instances with $|V| = 200$ from the 2nd DIMACS challenge on Maximum Clique, Graph Coloring, and Satisfiability (**DIMACS2**). Most of them are still employed as test-bed by state-of-the-art exact MCP algorithms.

Set C – Real-world networks These instances have been taken from the [10th DIMACS Implementation Challenge - Graph Partitioning and Graph Clustering](#) and SNAP database. We consider instances with up to $\approx 100,000$ vertices and $\approx 3,200,000$ edges. The reported 30 networks all fall into one of the following categories:

- Social Networks (with vertices representing individuals) such as: *Interaction networks*, where the edges refer to interaction via message posts, for example e-mail, as in *ia-email-EU*; *Recommendation networks*, where the edges indicate communication in terms of a recommendation or an opinion (e.g., *rec-eachmovie* related to movie opinions); *Collaboration networks*, where edges represent scientific or other type of collaborations (e.g., *astro-ph*

refers to preprints in the astrophysics archive, and *cond-mat*-* refer to the condensed matter archive www.arxiv.com). In some other social networks, edges refer to relationships of different types, such as friendship (e.g. Facebook, *socfb*), who-trusts-whom network (*Soc-epinioins1*), bloggers-interactions (*soc-BlogCatalog*), etc.

- Technological networks: In this case vertices are routers and edges represent communications between them, as in *tech-internet-as*.
- Scientific computing networks: These networks are meshes that are derived from mathematical analysis in different fields of science, i.e. finite elements (*fe*-*), and other types (*sc*-* etc.).

7.3 Computational performance on random graphs of different size and densities

The purpose of this section is to demonstrate the effectiveness of the main components of CLIQUE-INTER and to determine the relative impact on the computational difficulty of the three main instance features, i.e., the number of vertices of a graph ($|V|$), the edge density of a graph ($\mu = \frac{2|E|}{|V| \cdot (|V|-1)}$) and the amount of available interdiction budget (k). To this end, we use 220 random instances of Set A and we set a *time limit* of 600 seconds of CPU time for each run.

Recall that CLIQUE-INTER is the default setting of our exact framework, fully exploiting all its components (see Section 7.1). We test four additional configurations of CLIQUE-INTER, removing some of its components and measuring their impact on the computational performance:

1. CLIQUE-INTER (no bounds): in this configuration, all the ingredients are turned on, except for the calculation of combinatorial upper and lower bounds (ℓ_{min} and ℓ_{max}).
2. CLIQUE-INTER (no maximality): in this configuration, all the ingredients are turned on, except that we did not make the cliques maximal before adding the corresponding CI cuts.
3. Basic CLIQUE-INTER with IMCQ: in this configuration all components are removed, except the use of IMCQ to separate CI cuts.
4. Basic CLIQUE-INTER with CPLEX: this configuration corresponds to the basic B&C approach in which CI cuts are separated using CPLEX as a black-box clique solver applied to the classical clique ILP formulation with constraints associated to non-neighbour vertices. This configuration corresponds to the general implementation for interdiction games under monotonicity as given in Fischetti et al. (2018).

In order to give a graphical representation of the relative performance of the different configurations of CLIQUE-INTER, we report a performance profile in Figure 3. For each instance, we compute a normalized time τ as the ratio of the computing time of the considered configuration over the minimum computing time for solving the instance to optimality. For each value of τ in the horizontal axis, the vertical axis reports the percentage of the instances for which the corresponding configuration spent at most τ times the computing time of the fastest configuration. The curves start from the percentage of instances in which the corresponding configuration is

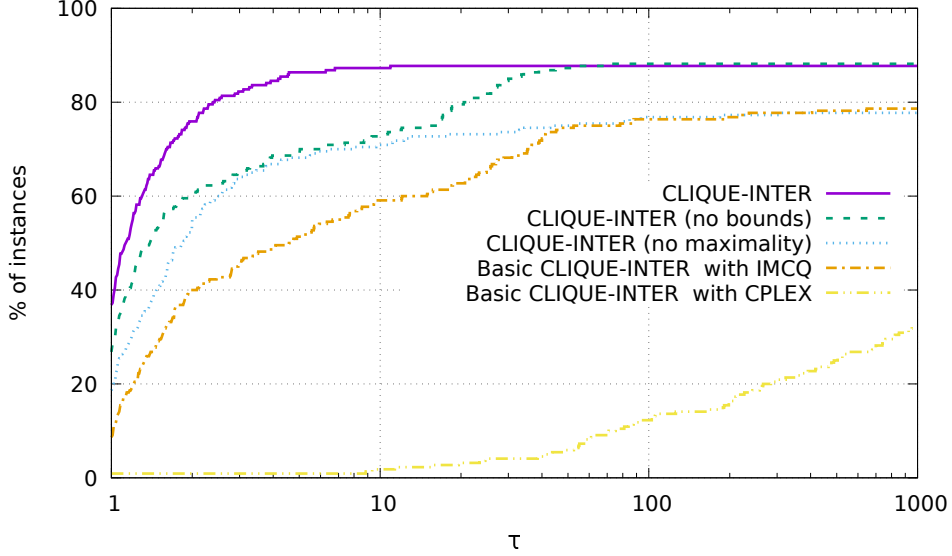


Figure 3: Performance profile of the different configurations of CLIQUE-INTER.

the fastest and at the right end of the chart, we can read the percentage of instances solved by a specific CLIQUE-INTER configuration. The best performances are graphically represented by the curves in the upper part of Figure 3 and the horizontal axis is represented in logarithmic scale.

Figure 3 clearly shows that each of the components is necessary to achieve the best computational performance. CLIQUE-INTER is the fastest for 40% of the instances and it manages to solve to proven optimality almost 90% of the instances of Set A. From the figure, we can also see that the impact of separating CI cuts using IMCQ is crucial. Disabling the improvements based on $\ell_{\min}$ and $\ell_{\max}$ and not inserting “maximal” CI cuts have a remarkable detrimental effect, which is why the basic CLIQUE-INTER with IMCQ has a very poor computational performance. The computation cost of $\ell_{\min}$ and $\ell_{\max}$ is not negligible. For this reason, for the very small instances which are easily solvable, removing this computation can be somehow beneficial; for the most difficult instances, however, the update of the lower and upper bounds is crucial. The effects of the graph reduction technique for Set A were rather moderate (so we do not report these results separately), however this technique turns out to be inevitable for large-scale real-world networks (cf. Table 3), which is why we have decided to keep it in the default CLIQUE-INTER setting.

We now discuss the impact on the CPU time of the instance features using the best configuration of CLIQUE-INTER. In Figures 4 and 5, we report the box plots of the computational times grouping the instances by edge density (Figure 4) and by graph size ($|V|$) and by interdiction budget (k) (Figure 5). In these figures, we graphically represent the computing times (in logarithmic scale) through their quartiles; the lines extending vertically from the boxes indicate the variability outside the upper and lower quartiles. Finally the outliers are plotted as individual points. In the upper part of both figures, we report the number of instances solved to proven optimality ($\#OPT$) for each group of instances with similar characteristics and the number of instances solved by preprocessing ($\#PREP$), i.e., when $\ell_{\min} = \ell_{\max}$. In the latter case, an optimal solution is computed without tackling formulation (11)-(14). The number of instances

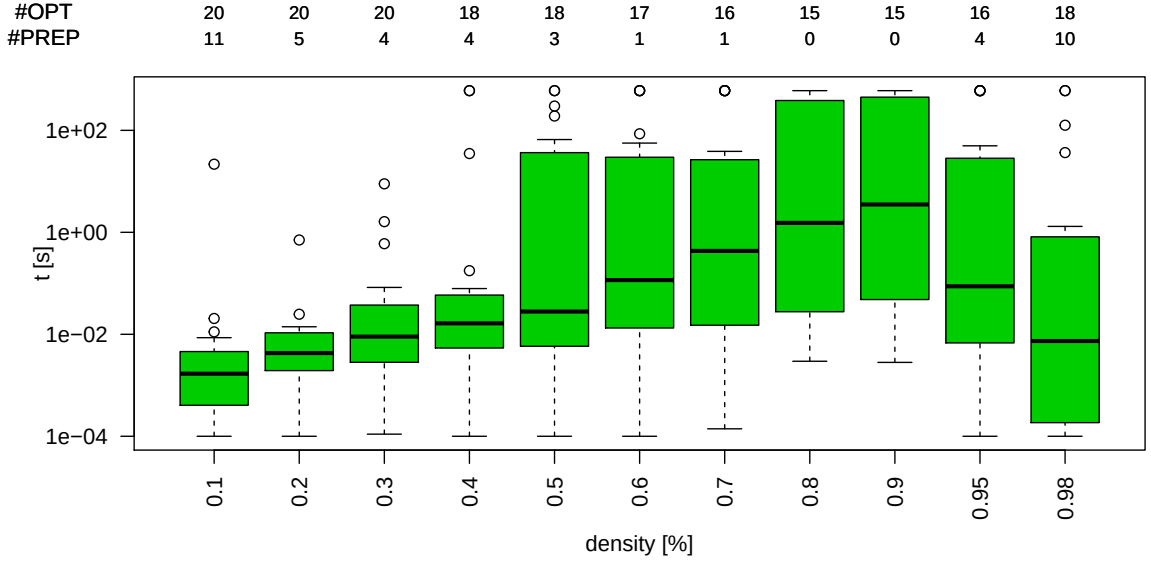


Figure 4: The total CPU times on the random graphs of Set A, grouped by the edge density.

solved by preprocessing is a proxy of the quality of both the CIP upper and lower bounds proposed in this paper.

In Figure 4, we have 11 groups of 20 instances each. We can observe that the computational time increases with the edge density up to a density value of 0.9 and then it starts to decrease. All instances with up to density 0.3 can be solved to proven optimality within the given time limit of 600 seconds by **CLIQUE-INTER**. The hardest instances are the ones with density 0.8 and 0.9, where **CLIQUE-INTER** is able to solve 15 out of 20 instances in each group. Several instances with low density can be solved by preprocessing, e.g., 11 for density 0.1 and 5 for density 0.2. None of the instances of density 0.8 and 0.9 can be solved by preprocessing. All instances with up to density 0.3 can be solved with an average CPU time less than 0.1 seconds. For all the other density classes the average CPU times of the solvable instances do not outmatch 100 seconds, but some instances can take up to several hundreds seconds.

In Figure 5, we have 20 groups of 11 instances each. As expected, we can observe that the computational time increases according to the number of vertices ($|V|$). In addition, the computational time is also directly correlated with the level of interdiction budget (k). All the instances with 50 and 75 vertices can be solved to proven optimality, independently of k . The figure shows that more instances can be solved by preprocessing for low levels of interdiction budget, e.g., for instances with 50 vertices, 7 instances can be solved by preprocessing when $k = \lceil 0.05 \cdot |V| \rceil$ and only 2 when $k = \lceil 0.4 \cdot |V| \rceil$. All the instances with 50 vertices are solved in less than 1 second and all the instances with 75 vertices are solved in less than 100 seconds. All instances with low values of k ($k = \lceil 0.05 \cdot |V| \rceil$ and $k = \lceil 0.1 \cdot |V| \rceil$) can be solved to proven optimality in less than 1 and 100 seconds, respectively.

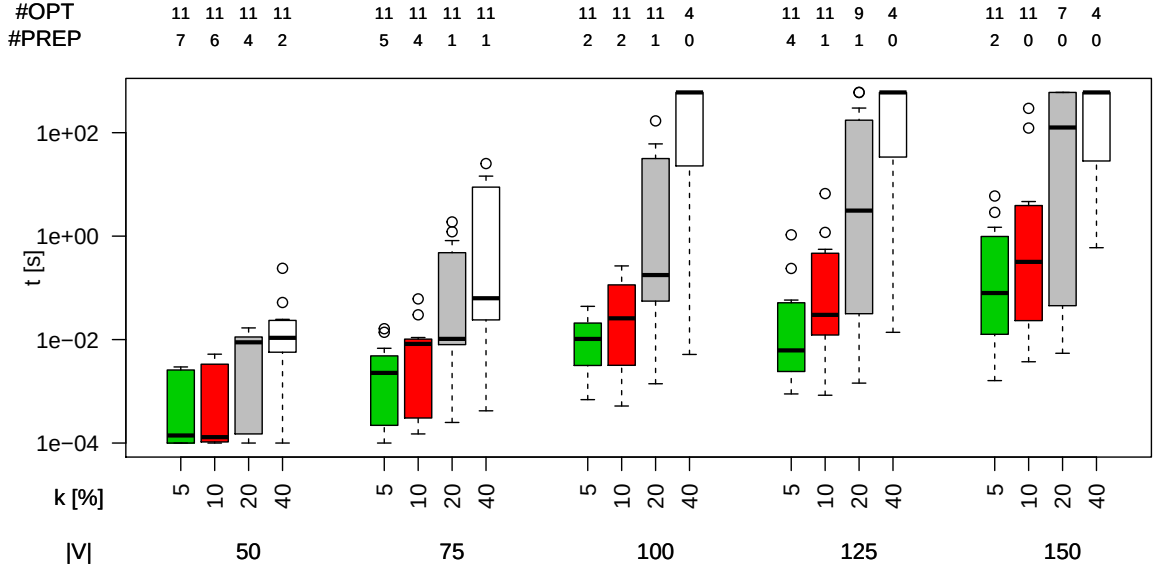


Figure 5: The total CPU times on the random graphs of Set A, grouped by the network size and the interdiction budget.

$ V $	#	CLIQUE-INTER				BILEVEL			
		# solved	time	exit gap	root gap	# solved	time	exit gap	root gap
50	44	44	0.01	-	0.16	28	68.58	6.44	8.50
75	44	44	1.45	-	0.41	14	120.19	9.47	10.91
100	44	37	9.30	1.00	0.98	7	164.42	12.65	13.11
125	44	35	13.43	1.33	1.20	2	135.33	13.88	14.73
150	44	33	27.23	1.91	1.43	1	397.52	16.42	16.39

Table 1: Comparison with state-of-the-art bilevel solver (BILEVEL) from Fischetti et al. (2017) and our approach (CLIQUE-INTER).

7.4 Comparison with a state-of-the-art bilevel solver

In Table 1, we compare the results of the state-of-the-art bilevel and interdiction game solver from Fischetti et al. (2017) (called BILEVEL) with our new approach CLIQUE-INTER. Each row corresponds to 44 instances of Set A grouped by the number of vertices $|V| \in \{50, 75, 100, 125, 150\}$. For this test we used the same time limit of 600 seconds for each run. For each of the two solvers, we report the following values: the number of solved instances per group (#solved), the average computing time in seconds for those instances that were solved to optimality, the average exit gap after the time limit is reached (considering only those instances which were *not* solved to optimality) and the average root gap (over all instances). The exit gap is calculated as $\text{exit gap} = \text{UB} - \lfloor \text{LB} \rfloor$, where UB refers to the global upper bound computed by the corresponding method, and LB refers to the global lower bound of the same method. In order to measure the quality of lower bounds at the root node of the B&C tree (denoted by LB_r), we compute them with respect to the best known solution (BKS) as $\text{root gap} = \text{BKS} - \lfloor \text{LB}_r \rfloor$.

Table 1 demonstrates that our new approach greatly outperforms the general-purpose bilevel solver of Fischetti et al. (2017) by several orders of magnitude. All instances with 50 and 75 vertices are solved to optimality in less than 2 seconds on average by CLIQUE-INTER, whereas the bilevel solver manages to solve only 2/3 and 1/3 of the instances, respectively. Similar behavior can be observed for larger instances, where, for example, for graphs with 150 vertices the bilevel solver manages to solve only a single instance in more than 5 minutes, whereas we solve 70% of them, in less than 1/2 minute on average. These results can be easily explained by the quality of root bounds: whereas the average absolute gaps at the root node for our approach are between 0.16 and 1.4, those of BILEVEL range between 8.5 and 16.4. This has a strong impact on the size of the B&C tree. For example, for 28 instances of size 50 solved by both approaches, CLIQUE-INTER requires an average number of 1.5 B&C nodes, compared to 1960 nodes required by BILEVEL.

7.5 Performance of CLIQUE-INTER on DIMACS2 graphs and large real-world networks

In Table 2, we show the performance of CLIQUE-INTER on the dense graphs of Set B. All these instances have 200 vertices and the table reports their density (μ), the size of maximum cliques ($\omega(G)$) and the CPU time necessary to compute it using IMCQ (time_ω). We test two levels of interdiction budget, i.e., $k = 20$ and $k = 40$. For these tests we set a time limit of 3600 seconds and T.L. is reported for the instances not solved to proven optimality. For each value of k , we report the value of the final lower and upper bounds (LB and UB), the total CPU time spent in solving formulation (11)-(14) and the values of the initial lower and upper bounds (ℓ_{min} and ℓ_{max}). In case an instance is solved to proven optimality then $\text{LB}=\text{UB}$, in case an instance is solved during the preprocessing $\ell_{min} = \ell_{max}$. In this latter case, we do not report the CPU time since it is not necessary to tackle formulation (11)-(14), and the time necessary to compute ℓ_{min} and ℓ_{max} is negligible. The time taken by IMCQ to find $\omega(G)$ for the instances of Set B is often negligible, except for `sanr200_0.9`, where it takes 1.9 seconds. CLIQUE-INTER is able to solve 12 and 8 instances out of 14 instances, for $k = 20$ and $k = 40$, respectively. From the table, we can see that the gap between ℓ_{min} and ℓ_{max} is small for all instances except in a few cases, e.g., for `san200_0.9_1` and `san200_0.9_2` and $k = 40$ where the gap is 10 and 8, respectively. In most of the remaining instances the initial gap is less than 3, showing that also for this set of instances ℓ_{min} and ℓ_{max} provide good-quality lower and upper bounds. For $k = 20$, three instances can be solved to proven optimality by preprocessing alone and, for $k = 40$, two instances are solved by preprocessing. As already observed on the instance Set A, increasing the level of budget increases the computational difficulty as well.

In Table 3, we show the performance of CLIQUE-INTER on the large real-world networks of Set C. As discussed in Section 7.2, this set is composed by 30 sparse graphs with up to 100K vertices and 3200K edges. The table reports for each instance the number of vertices ($|V|$), the number of edges ($|E|$), the size of maximum clique ($\omega(G)$) and the CPU time necessary to compute it using IMCQ (time_ω). For these tests, we used a time limit of 3600 seconds for each run. Since these graphs are large, to test realistic levels of interdiction budget, we set $k = \lceil 0.005 \cdot |V| \rceil$ and

	μ	$\omega(G)$	time_ω	CLIQUE-INTER $k = 20$					CLIQUE-INTER $k = 40$				
				LB	UB	time	$\ell_{\min}$	$\ell_{\max}$	LB	UB	time	$\ell_{\min}$	$\ell_{\max}$
brock200_1	0.75	21	0.2	18	18	938.2	16	18	15	17	T.L.	13	17
brock200_2	0.50	12	0.0	9	9	0.1	8	10	8	9	T.L.	7	9
brock200_3	0.61	15	0.0	12	12	1.0	11	13	11	11	160.6	9	12
brock200_4	0.66	17	0.0	14	14	2421.8	12	15	12	13	T.L.	10	13
c-fat200-1	0.08	12	0.0	10	10	-	10	10	9	9	-	9	9
c-fat200-2	0.16	24	0.0	20	20	-	20	20	18	18	-	18	18
c-fat200-5	0.43	58	0.0	52	52	0.0	51	52	46	46	0.0	44	46
san200_0.7_1	0.70	30	0.0	17	17	5.4	16	18	15	15	134.4	14	17
san200_0.7_2	0.70	18	0.0	14	14	16.7	13	15	12	12	5.6	11	15
san200_0.9_1	0.90	70	0.0	50	50	-	50	50	40	40	13.3	39	49
san200_0.9_2	0.90	60	0.1	41	41	3.2	41	42	34	34	2266.9	33	41
san200_0.9_3	0.90	44	0.0	33	34	T.L.	32	37	28	31	T.L.	26	34
sanr200_0.7	0.70	18	0.1	15	15	29.2	14	16	13	14	T.L.	11	15
sanr200_0.9	0.90	42	1.9	33	35	T.L.	31	35	28	32	T.L.	25	33
gen200_p0.9_44	0.90	44	0.1	34	34	674.4	32	38	29	31	T.L.	26	36
gen200_p0.9_55	0.90	55	0.1	38	38	62.4	37	41	32	33	T.L.	29	40

Table 2: Computational results obtained by the CLIQUE-INTER on the instances with $|V| = 200$ from the 2nd DIMACS Challenge [DIMACS2](#).

$k = \lceil 0.01 \cdot |V| \rceil$. For this set of instances the time to compute $\ell_{\min}$ is not always negligible and we report it in the column time_p . As explained in Section 4.1, $\ell_{\min}$ is computed starting from a set $\mathcal{Q}_p = (C_1, \dots, C_p)$ of p vertex-disjoint cliques of G and the minimum p , which provides the best $\ell_{\min}$, is determined by the level of interdiction budget k . In the table, we only report the CPU time necessary to compute $\ell_{\min}$ for $k = \lceil 0.01 \cdot |V| \rceil$, the time to compute $\ell_{\min}$ for the $k = \lceil 0.001 \cdot |V| \rceil$ is smaller. The table reports for each instance the optimal solution value (OPT^*) and, in case the time limit is reached, we report T.L. and OPT^* is the best known UB. As for Table 2, the total CPU time spent in solving formulation (11)-(14) is reported in the column “time”. In the column $|V_{prep}|$, we report the number of vertices that can be removed from the graph, as explained in Section 4.2. We report the value of the initial upper and lower bound $\ell_{\min}$ and $\ell_{\max}$ and the number of CI cuts separated and the number of B&C nodes explored solving formulation (11)-(14). CLIQUE-INTER is able to solve to proven optimality 28 out of 30 instances for both levels of interdiction budget. In the preprocessing phase, 6 and 4 instances are solved for $k = \lceil 0.005 \cdot |V| \rceil$ and $k = \lceil 0.01 \cdot |V| \rceil$, respectively. The gap between $\ell_{\min}$ and $\ell_{\max}$ is relatively small also for this set of instances, with a few exceptions, e.g, for *socfb-UF* and $k = \lceil 0.01 \cdot |V| \rceil$ the gap is 15 or for *socfb-Ullinois* and $k = \lceil 0.01 \cdot |V| \rceil$ the gap is 11. The graph reduction technique is crucial for these set of large instances. The table shows that huge reductions can be achieved, i.e., if we compare the size of the initial set of vertices $|V|$ and the number of vertices that can be discarded $|V_{prep}|$, we can see that formulation (11)-(14) is actually solved on a small fraction of vertices. This reduction is more effective for small values of interdiction budget k . Finally, the columns “cuts” and “nodes” reveal that the number of CI cuts separated and B&C nodes explored is often small. Only in some cases, several hundreds of cuts are separated using IMCQ and the number of explored nodes is typically smaller than 500.

In some rare cases the number of explored nodes can be high, as in *Slashdot0902*, which requires 14105 nodes.

7.6 Clique-interdiction curve of a graph

We define the *resilience of (social) networks with respect to clique-interdiction attacks* as the relative decrease of the size of the maximum clique in function of the interdiction budget. This can also be seen as the ability of the network to survive a clique-interdiction attack.

We plot the size of an optimal CIP solution (as a percentage of the size of a maximum clique on the original graph) for different values of the interdiction budget k . Figure 6 depicts this relation for two quite different types of networks: very sparse social networks (taken from the DIMACS10 data set) versus very dense graphs (taken from the DIMACS2 data set). On the x axis we plot the percentage of the total number vertices (representing the interdiction budget k). On the y axis, we provide the relative size of the optimal CIP solution compared to the value of $\omega(G)$. Not surprisingly, the obtained results indicate that significantly higher interdiction budget is needed for dense networks. With the interdiction budget of only 1% of vertices, one can reduce the size of the maximum clique between 40% and 95% in case of social networks (see *astro-ph* and *memplus*), whereas dense networks are much more resistant to this kind of attacks. With the 1% of interdiction budget, the size of the maximum clique can be reduced only by a few percent in most of the cases shown in Figure 6.

The obtained results, both in terms of CPU times and the size of networks that can be solved to optimality, indicate that our solver can be used as a powerful decision making tool for analyzing and assessing the network resilience against clique-interdiction attacks. Measuring the clique-interdiction number can provide to decision makers an important information about the structure of the network concerning its densely connected components. Our exact solver can also be used for simulation purposes, to find out which vertices need to be protected (in case of a clique-interdiction attack).

8 Extensions and conclusions

In this article we studied the interdiction problem in a network in which the leader chooses up to k vertices to delete, so as to minimize the clique number determined by the follower in the resulting network. We studied the problem complexity for special graphs, and we derived a single-level ILP formulation with an exponential number of constraints called clique-interdiction cuts. In a polyhedral study of the underlying polytope, we provided necessary and sufficient conditions for these cuts to be facet defining and we designed effective lifting procedures. The separation of these cuts required development of an efficient exact solver for the maximum clique problem, which we tailored for the clique interdiction problem. The deep understanding of the underlying problem allowed us to derive tight combinatorial lower and upper bounds, along with an efficient preprocessing phase for drastically reducing the problem size. Our framework is the first one being able to solve to proven optimality real-world large-scale (social) networks from various publicly available resources, thus, drastically outperforming the state-of-the-art general

purpose bilevel solver from [Fischetti et al. \(2017\)](#). This shows that significant improvements (in terms of the size of input networks and computing times) can be achieved by tailored exact algorithms for particular classes of interdiction problems. The code will be made available online (after the revision process is finished), to help decision makers analyze the resilience of networks to maintain a large clique after a clique-interdiction attack.

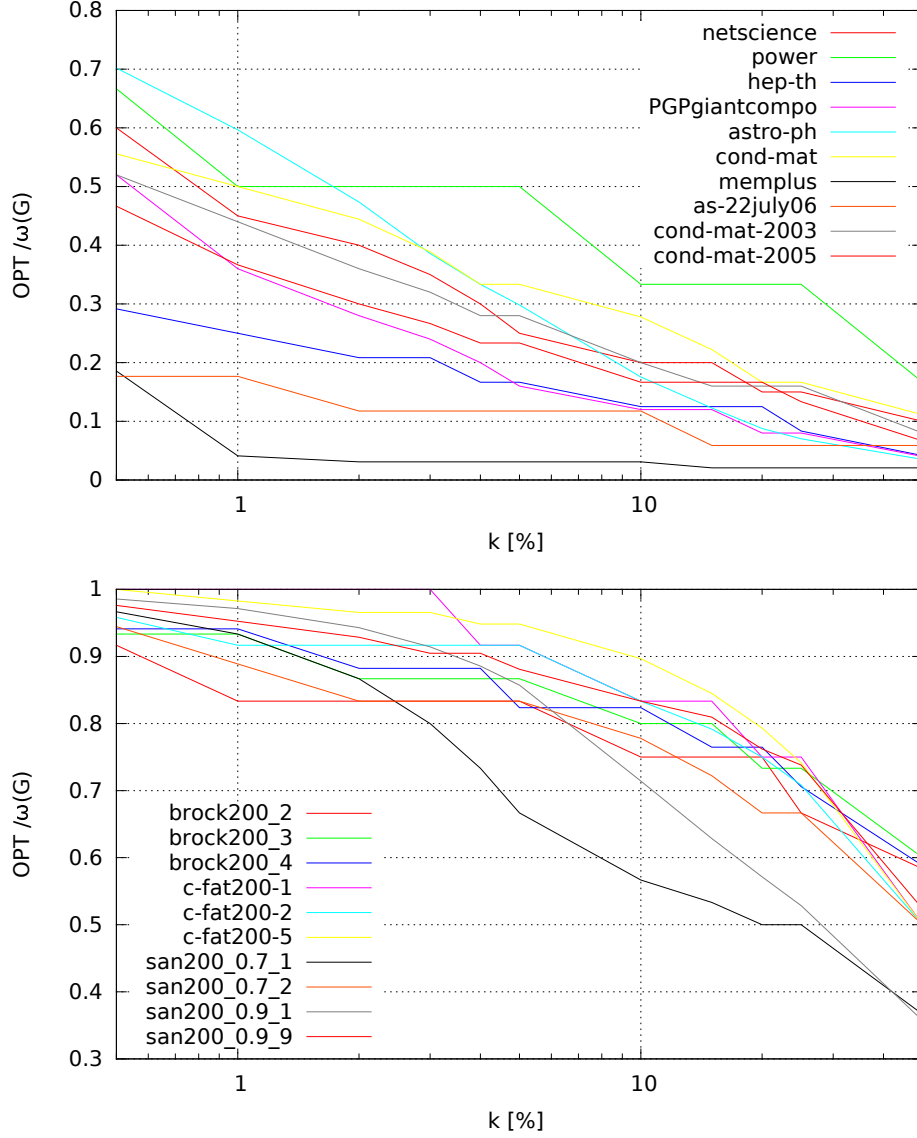


Figure 6: Clique-interdiction curves for social networks (up) and dense graphs (down).

	CLIQUE-INTER $k = \lceil 0.005 \cdot V \rceil$						CLIQUE-INTER $k = \lceil 0.01 \cdot V \rceil$												
	$ V $	$ E $	$\omega(G)$	time_ω	time_p	OPT*	time	$ V_{\text{prep}} $	$\ell_{\min}$	$\ell_{\max}$	cuts	nodes	OPT*	time	$ V_{\text{prep}} $	$\ell_{\min}$	$\ell_{\max}$	cuts	nodes
PGPgiantcompo	10,680	24,316	25	0.0	0.0	13	0.0	10,409	13	16	11	0	9	0.0	10,047	9	12	31	2
astro-ph	16,706	121,251	57	0.0	0.1	40	0.0	16,206	40	41	0	0	34	0.0	15,872	34	35	4	0
memplus	17,758	54,196	97	0.0	0.1	18	0.0	17,569	18	32	1	0	4	0.0	15,030	4	5	9	0
as-22july06	22,963	48,436	17	0.0	0.7	3	-	-	3	3	-	-	3	-	-	3	3	-	-
cond-mat-2005	40,421	175,691	30	0.0	0.8	14	0.1	38,270	13	15	12	0	11	0.3	36,866	11	13	47	3
Cit-HepPh	34,546	420,877	19	0.0	4.3	11	2.8	15,493	11	14	60	0	9	52.4	12,501	9	13	525	2188
Soc-Epinions1	75,879	405,740	23	0.1	17.6	8	62.7	64,859	8	12	339	138	6	114.2	61,968	6	9	719	126
Slashdot0902	82,168	504,230	27	0.0	37.9	4	921.5	50,736	4	6	2035	14105	4	92.5	50,736	4	5	380	0
socfb-UIllinois	30,795	1,264,421	57	0.5	7.9	33	24.4	10,456	33	42	57	13	27	41.6	8290	27	38	106	44
ia-email-EU	32,430	54,397	12	0.0	0.5	4	0.6	30,375	4	6	178	47	3	0.5	29,212	3	4	165	3
rgg_n_2_15_s0	32,768	160,240	13	0.0	0.7	10	-	-	10	10	-	-	9	0.2	30,848	9	10	35	0
ia-enron-large	33,696	180,811	20	0.0	1.5	9	2.2	27,791	8	12	81	15	7	29.5	26,651	7	10	957	2827
socfb-UF	35,111	1,465,654	55	0.3	7.2	37	17.8	14,264	37	48	74	18	29	87.8	10,708	29	44	272	234
socfb-Texas84	36,364	1,590,651	51	0.3	8.5	30	24.6	10,706	30	43	66	19	25	74.3	8,704	25	36	168	104
tech-internet-as	40,164	85,123	16	0.0	2.8	3	1.4	31,783	3	4	152	4	3	-	-	3	3	-	-
fe-body	45,087	163,734	6	0.1	5.4	4	1.8	2,259	4	5	27	0	4	1.8	2259	4	5	27	0
sc-nasasrb	54,870	1,311,227	24	0.1	7.3	24	-	-	24	24	-	-	23	145.5	1,195	23	24	1249	254
soc-brightkite	56,739	212,945	37	0.0	3.7	7	15.6	47,159	7	11	482	314	6	11.2	44,919	6	8	322	18
soc-loc-brightkite	58,228	214,078	37	0.0	4.0	7	11.9	48,640	7	11	390	169	6	12.6	46,384	6	8	347	18
tech-p2p-gnutella	62,561	147,878	4	0.1	15.4	3	-	-	3	3	-	-	3	-	-	3	3	-	-
delaunay_n16	65,536	196,575	4	0.2	70.9	4	-	-	4	4	-	-	4	-	-	4	4	-	-
rgg_n_2_16_s0	65,536	342,127	14	0.0	5.1	10	0.3	63,637	10	11	26	0	9	1.5	59,534	9	10	57	0
soc-themarker	69,413	1,644,843	22	2.1	68.0	8	T.L.	35,678	7	10	1685	4806	6	T.L.	31,101	5	8	2124	1311
rec-eachmovie	74,424	1,634,743	12	0.7	18.5	3	-	-	3	3	-	-	2	367.3	13669	2	3	3854	569
fe-tooth	78,136	452,591	5	0.5	58.0	4	18.9	7	4	5	30	0	4	19.0	7	4	5	30	0
sc-pkustk11	87,804	2,565,054	36	1.1	54.9	24	70.7	2,712	24	30	62	4	24	57.1	2,712	24	30	51	0
soc-BlogCatalog	88,784	2,093,195	45	11.7	149.8	11	T.L.	51,607	8	12	1327	1376	8	T.L.	46,240	6	9	2281	1637
ia-wiki-Talk	92,117	360,767	15	0.2	16.6	5	49.2	72,678	4	6	345	23	4	87.4	72,678	4	5	711	14
sc-pkustk13	94,893	3,260,967	36	1.3	310.0	35	724.9	2,360	35	36	535	80	34	879.2	2,354	34	36	613	38
fe_rotor	99,617	662,431	5	1.0	182.8	4	200.5	0	4	5	184	0	4	200.2	0	4	5	184	0

Table 3: Computational results obtained by the CLIQUE-INTER on selected large-scale real-world networks.

Extensions In some applications where the concept of clique may be too restrictive to define a community, *clique-relaxations* can be used instead, see (Balasundaram et al., 2011; Pattillo et al., 2013). Clique-relaxations are obtained by relaxing some of the clique defining properties, and our exact solution framework can be extended to the interdiction problems in which the follower solves the maximum relaxed clique problem, as long as the latter one satisfies the hereditary property. Relaxed cliques with hereditary properties are obtained by relaxing: the degree of the vertices (*s*-plex), the distance between the vertices (*s*-clique), the density of the edges (*s*-defective clique) and the connectivity between the vertices (*s*-bundle). Precise definition of these problems can be found in e.g., Pattillo et al. (2013). One can show that our single-level reformulation (and accordingly the branch-and-cut algorithm) can be extended for solving this class of interdiction problems. The key element of the B&C implementation would be an efficient exact solver for the maximum relaxed clique problem at hand. We point out however that the theory and methodology derived in this paper does not directly carry over to solving interdiction problem in which the follower maximizes a *weighted* hereditary problem on a network. These problems deserve a special attention and will be subject of our future studies.

Other interesting problem extensions concern the introduction of multiple leaders (that could act in a collaborative or competitive way), multiple followers, or both (see, e.g., Sinha et al. (2014)). Similarly, a multi-period setting could be worthy of investigation, as well as allowing incomplete information (like, e.g., in Stackelberg security games, Guo et al. (2016); Lagos et al. (2017)).

Finally, it would be interesting to combine other centrality measures with the clique number for measuring the cohesiveness of a network. That way, one could capture vertices which are the most vital ones with respect to different measures simultaneously (as for example in Rysz et al. (2018), where the authors combine maximum cliques with the betweenness centrality). A very interesting line of research would be to take into consideration the cohesiveness of a network and, at same time, its connectivity. Studying the most vital vertices with respect to such new measures of cohesiveness would certainly provide some new and relevant insights for the analysis of (social) networks.

Acknowledgment

Valuable comments of four anonymous reviewers are highly appreciated.

Ivana Ljubić is partially funded by the Vienna Science and Technology Fund (WWTF) through project ICT15-014. Pablo San Segundo is funded by the Spanish Ministry of Economy and Competitiveness (grants DPI 2014-53525-C3-1-R, DPI2017-86915-C3-3-R).

References

References

10th DIMACS Implementation Challenge - Graph Partitioning and Graph Clustering, Nov 20,

2017. URL <https://www.cc.gatech.edu/dimacs10/>.
- 2017 Terrorist Attacks, Nov 20, 2017. URL <http://storymaps.esri.com/stories/terrorist-attacks/>.
- D. Adjashvili, A. Baggio, and R. Zenklusen. Firefighting on trees beyond integrality gaps. In P. N. Klein, editor, *Proceedings of the Twenty-Eighth Annual ACM-SIAM Symposium on Discrete Algorithms, SODA 2017, Barcelona, Spain, Hotel Porta Fira, January 16-19*, pages 2364–2383. SIAM, 2017.
- N. Assimakopoulos. A network interdiction model for hospital infection control. *Computers in Biology and Medicine*, 17(6):413 – 422, 1987.
- A. Baggio. *Towards Optimal Approximations for Firefighting and Related Problems*. PhD thesis, ETH Zürich, 2016.
- E. Balas and C. S. Yu. Finding a maximum clique in an arbitrary graph. *SIAM J. Comput*, 15(4):1054–1068, 1986.
- B. Balasundaram, S. Butenko, and I. V. Hicks. Clique relaxations in social network analysis: The maximum k -plex problem. *Operations Research*, 59(1):133–142, 2011.
- V. Batagelj and M. Zaversnik. Fast algorithms for determining (generalized) core groups in social networks. *Adv. Data Analysis and Classification*, 5(2):129–145, 2011.
- C. Bazgan, S. Toubaline, and Z. Tuza. The most vital nodes with respect to independent set and vertex cover. *Discrete Applied Mathematics*, 159:1933 – 1946, 2011.
- N. Berry, T. Ko, T. Moy, J. Smrcka, J. Turnley, and B. Wu. Emergent Clique Formation in Terrorist Recruitment. In *The AAAI-04 Workshop on Agent Organizations: Theory and Practice, July 25, 2004, San José, California*. National Conference on Artificial Intelligence, 2004.
- S. Borgatti, M. Everett, and J. Johnson. *Analyzing Social Networks*. Sage, 2018.
- P. Cappanera and M. P. Scaparra. Optimal allocation of protective resources in shortest-path networks. *Transportation Science*, 45(1):64–80, 2011.
- A. Caprara, M. Carvalho, A. Lodi, and G. J. Woeginger. Bilevel knapsack with interdiction constraints. *INFORMS Journal on Computing*, 28(2):319–333, 2016.
- H. Chen, W. Chung, J. J. Xu, G. Wang, Y. Qin, and M. Chau. Crime data mining: A general framework and some examples. *Computer*, 37(4):50–56, 2004.
- K. Cormican, D. Morton, and K. Wood. Stochastic network interdiction. *Operations Research*, 46(2):184 – 197, 1998.
- S. Dempe. Bilevel optimization: theory, algorithms and applications. *Optimization Online*, 2018. Preprint 2018-11, Fakultät für Mathematik und Informatik, TU Bergakademie Freiberg.

- DIMACS2. 2nd DIMACS Implementation Challenge - NP Hard Problems: Maximum Clique, Graph Coloring, and Satisfiability, Nov 20, 2017. URL <http://dimacs.rutgers.edu/Challenges/>.
- M. Fischetti, I. Ljubić, M. Monaci, and M. Sinnl. A new general-purpose algorithm for mixed-integer bilevel linear programs. *Operations Research*, 65(60):1615–1637, 2017.
- M. Fischetti, I. Ljubić, M. Monaci, and M. Sinnl. Interdiction games under monotonicity. *INFORMS Journal on Computing*, 2018. To appear.
- F. Furini, I. Ljubić, E. Malaguti, and P. Paronuzzi. On integer and bilevel formulations for the k -vertex cut problem. submitted.
- C. García-Martínez, C. Blum, F. Rodriguez, and M. Lozano. The firefighter problem: Empirical results on random graphs. *Computers & Operations Research*, 60:55 – 66, 2015.
- M. R. Garey and D. S. Johnson. *Computers and Intractability: A Guide to the Theory of NP-completeness*. W.H. Freeman, San Francisco, 1979.
- Q. Guo, B. An, Y. Vorobeychik, L. Tran-Thanh, J. Gan, and C. Miao. Coalitional security games. In *Proceedings of the 2016 International Conference on Autonomous Agents & Multiagent Systems, AAMAS '16*, pages 159–167, Richland, SC, 2016. International Foundation for Autonomous Agents and Multiagent Systems.
- D. Kempe, J. Kleinberg, and É. Tardos. Influential nodes in a diffusion model for social networks. In L. Caires, G. F. Italiano, L. Monteiro, C. Palamidessi, and M. Yung, editors, *Automata, Languages and Programming*, pages 1127–1138, Berlin, Heidelberg, 2005. Springer Berlin Heidelberg. ISBN 978-3-540-31691-6.
- H. Kim and N. Feamster. Improving network management with software defined networking. *IEEE Communications Magazine*, 51(2):114–119, 2013.
- K.-I. Ko and C.-L. Lin. *On the Complexity of Min-Max Optimization Problems and their Approximation*, pages 219–239. Springer US, Boston, MA, 1995.
- F. Lagos, F. Ordóñez, and M. Labbé. A branch and price algorithm for a Stackelberg security game. *Computers & Industrial Engineering*, 111:216–227, 2017.
- M. Lalou, M. A. Tahraoui, and H. Kheddouci. The critical node detection problem in networks: A survey. *Computer Science Review*, 28:92 – 117, 2018. ISSN 1574-0137.
- C. Li, H. Jiang, and F. Manyà. On minimization of the number of branches in branch-and-bound algorithms for the maximum clique problem. *Computers & OR*, 84:1–15, 2017.
- L. Lozano and J. Smith. A value-function-based exact approach for the bilevel mixed integer programming problem. *Operations Research*, 65(3):768–786, 2017.
- F. Mahdavi Pajouh, V. Boginski, and E. L. Pasiliao. Minimum vertex blocker clique problem. *Networks*, 64(1):48–64, 2014.

- J. Moody and J. Coleman. *Clustering and Cohesion in Networks: Concepts and Measures*, pages 906–912. 2015.
- J. Moody and D. R. White. Structural cohesion and embeddedness: A hierarchical concept of social groups. *American Sociological Review*, 68(1):103–127, 2003.
- G. L. Nemhauser and L. A. Wolsey. *Integer and Combinatorial Optimization*, volume 18. Wiley New York, 1988.
- C. Papadimitriou. *Computational Complexity*. Addison-Wesley, Reading, MA, 1994.
- J. Pattillo, N. Youssef, and S. Butenko. On clique relaxation models in network analysis. *European Journal of Operational Research*, 226(1):9–18, 2013.
- H. D. Ratliff, G. T. Sicilia, and S. H. Lubore. Finding the n most vital links in flow networks. *Management Science*, 21(5):531–539, 1975.
- R. B. Rothenberg, J. J. Potterat, and D. E. Woodhouse. Personal risk taking and the spread of disease: Beyond core groups. *The Journal of Infectious Diseases*, 174:S144–S149, 1996.
- V. Rutenburg. Complexity classification of truth maintenance systems. In C. Choffrut and M. Jantzen, editors, *STACS 91: 8th Annual Symposium on Theoretical Aspects of Computer Science Hamburg, Germany, February 14–16, 1991 Proceedings*, pages 372–383. Springer Berlin Heidelberg, 1991.
- V. Rutenburg. Propositional truth maintenance systems: Classification and complexity analysis. *Annals of Mathematics and Artificial Intelligence*, 10(3):207–231, 1994.
- M. Rysz, F. M. Pajouh, and E. L. Pasiliao. Finding clique clusters with the highest betweenness centrality. *European Journal of Operational Research*, 271(1):155–164, 2018.
- M. Sageman. *Understanding Terrorist Networks*. University of Pennsylvania Press, Philadelphia, PA, USA, 2004.
- R. J. Sampson and W. B. Groves. Community structure and crime: Testing social-disorganization theory. *American Journal of Sociology*, 94(4):774–802, 1989.
- P. San Segundo and C. Tapia. Relaxed approximate coloring in exact maximum clique search. *Computers & OR*, 44:185–192, 2014.
- P. San Segundo, D. Rodriguez-Losada, and A. Jimenez. An exact bit-parallel algorithm for the maximum clique problem. *Computers & OR*, 38(2):571–581, 2011.
- P. San Segundo, F. Matia, D. Rodriguez-Losada, and M. Hernando. An improved bit parallel exact maximum clique algorithm. *Optimization Letters*, 7(3):467–479, 2013.
- P. San Segundo, A. Nikolaev, and M. Batsyn. Infra-chromatic bound for exact maximum clique search. *Computers & OR*, 64:293–303, 2015.

- P. San Segundo, A. Lopez, and P. M. Pardalos. A new exact maximum clique algorithm for large and massive sparse graphs. *Computers & OR*, 66:81–94, 2016a.
- P. San Segundo, A. Nikolaev, M. Batsyn, and P. M. Pardalos. Improved infra-chromatic bound for exact maximum clique search. *Informatica, Lith. Acad. Sci.*, 27(2):463–487, 2016b.
- A. Sinha, P. Malo, A. Frantsev, and K. Deb. Finding optimal strategies in a multi-period multi-leader–follower Stackelberg game using an evolutionary algorithm. *Computers & Operations Research*, 41:374 – 385, 2014.
- L. V. Snyder, Z. Atan, P. Peng, Y. Rong, A. J. Schmitt, and B. Sinsoysal. OR/MS models for supply chain disruptions: a review. *IIE Transactions*, 48(2):89–109, 2016.
- Y. Song and S. Shen. Risk averse shortest path interdiction. *INFORMS Journal on Computing*, 28(3):527–539, 2016.
- Y. Tang, J.-P. P. Richard, and J. C. Smith. A class of algorithms for mixed-integer bilevel min–max optimization. *Journal of Global Optimization*, 66(2):225–262, 2016.
- Z. Wang, Y. Yin, and B. An. Computing optimal monitoring strategy for detecting terrorist plots. In *Proceedings of the Thirtieth AAAI Conference on Artificial Intelligence*, AAAI’16, pages 637–643. AAAI Press, 2016.
- A. Washburn and K. Wood. Two-person zero-sum games for network interdiction. *Operations Research*, 43(2):243–251, 1995.
- R. K. Wood. *Bilevel Network Interdiction Models: Formulations and Solutions*. John Wiley & Sons, Inc., 2010.